

Time-Dependent Decision Making in Heuristic Chess Search

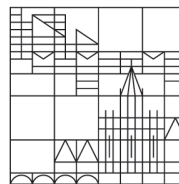
Bachelor Thesis

submitted by

Marvin Becker (01/1365135)

at the

Universität
Konstanz



Algorithmics

Department of Computer and Information Science

Advisor: Prof. Dr. Sabine Storandt

Reviewer: Prof. Dr. Sabine Storandt
Prof. Dr. Michael Grossniklaus

Konstanz, 2026-09-01



Abstract

Chess engines rely on heuristic search to cope with the exponential growth of the game tree, and under a clock the decisive question is how much additional computation is actually worth. This thesis investigates where the sweet spot between invested time and decision quality lies: how much extra search time translates into better moves, and from which point on further computation no longer changes the decision.

A complete engine is implemented from scratch in C++ on a classical mailbox board representation, combining alpha-beta search, transposition tables, quiescence search, heuristic move ordering and a two-stage time management. On this baseline three established refinements are implemented and evaluated: Principal Variation Search, null-move pruning, and the reuse of previous search trees through a persistent transposition table. The evaluation uses a suite of 309 positions across budgets from 10 ms to 5 s, with move quality scored as centipawn loss against a strong reference engine, and a 400-game match against a strength-limited opponent, won by 205 ± 38 Elo, as an external check.

All three refinements deliver depth as intended: at a one-second budget null-move pruning gains a full ply, Principal Variation Search 0.23 and tree reuse 0.25 ply, each with 95 % confidence intervals excluding zero. For none of them could a corresponding improvement in move quality be detected. Two observations explain this. Decisions settle early, as three quarters of all positions have fixed their move by the depth the engine already reaches. And the residual error is highly concentrated: the median centipawn loss is 2 while the mean is 52, with fifteen positions accounting for nearly half of that mean. These are predominantly sparse endgames, and they respond to additional depth far more weakly than the rest of the suite.

The value of additional computation is therefore highly heterogeneous across positions. Under an explicit criterion, no further improvement of typical move quality is detectable beyond roughly one second per move on this suite, while average quality keeps falling through that difficult minority. This is consistent with limitations of the static evaluation contributing substantially to the residual error.



Contents

Abstract	i
1 Introduction	1
2 Related Work	3
3 Preliminaries	5
3.1 Chess as a Game-Theoretic Model	5
3.2 Search Problem Formulation	5
3.3 Minimax, Negamax, Quiescence and Alpha-Beta Pruning	6
3.4 Heuristic Evaluation	9
3.5 Time-Constrained Search	9
4 Algorithm & Implementation	13
4.1 Engine Architecture	13
4.2 Search Algorithms	16
4.3 Time Management and Constraints	23
4.4 Evaluation Function	26
5 Experimental Evaluation	31
5.1 Research Objectives	31
5.2 Experimental Setup	32
5.3 Playing Strength (Engine versus Engine)	34
5.4 Effect of Null-Move Pruning	35
5.5 Transposition Table Size	37
5.6 Time-to-Quality and Depth-versus-Time	38
5.7 Distribution of Move Quality	39
5.8 Effect of Principal Variation Search	42
5.9 Search Tree Reuse	43
5.10 Move Stability	45
6 Conclusions and Future Work	49
6.1 Depth and Decision Quality Are Not Proportional	49
6.2 Where the Search Is Actually Limited	49
6.3 Implications for Time-Managed Search	51
6.4 Limitations	51
6.5 Reflection	52
6.6 Future Work	52
6.7 Summary	53

References	55
----------------------	----



1 Introduction

Chess has long been regarded as a canonical benchmark problem in artificial intelligence and algorithm engineering [17, 7, 15]. Despite its simple and fully specified rules, the game exhibits an enormous search space caused by its high branching factor and strategic depth. As a consequence, exhaustive search of the complete game tree is computationally infeasible, even with modern hardware, and chess engines must rely on heuristic search techniques to make strong decisions under limited computational resources.

At the core of most classical chess engines lies a depth-limited game tree search based on Minimax or Negamax, enhanced by Alpha-Beta pruning. Over the years, a variety of improvements have been proposed to increase search efficiency, including iterative deepening, transposition tables, quiescence search, and heuristic move ordering [9]. Among these techniques, iterative deepening [8] has become a standard approach, particularly in time-controlled settings, as it allows the engine to gradually refine its decision while always maintaining a valid best move.

The primary motivation of this work is to understand how a chess engine should spend its limited time. More search time generally yields deeper searches and better moves, but the relationship is far from linear: at some point the chosen move stabilizes and additional computation no longer changes the decision. At the same time, search refinements such as null-move pruning, Principal Variation Search and the reuse of previous search trees change how efficiently a given time budget is converted into depth and decision quality. This leads to the central research question of this thesis:

Where lies the sweet spot between invested search time and decision quality in heuristic chess search, and how do search refinements such as null-move pruning, Principal Variation Search and search tree reuse shift this trade-off?

To make this question measurable, the thesis examines five concrete aspects: (i) *move stability*, i.e., the depth at which the best move stabilizes and deeper search no longer changes the decision, (ii) the *time-to-quality curve*, i.e., how move quality changes as the time limit increases, (iii) the *depth and quality* gained by null-move pruning and by search tree reuse, (iv) the *contribution of Principal Variation Search* over a plain full-window search, together with a parameter sweep that determines which combination of minimum scouting depth and number of fully searched moves yields the best ratio of cost reduction to move quality, and (v) the *distribution* of move quality over the suite, which reveals where the remaining errors are concentrated.

To address this question, a complete chess engine is implemented from scratch in C++. The engine employs a classical mailbox-based board representation rather than highly optimized bitboard techniques, which keeps the impact of search algorithms and time management observable in isolation from low-level optimizations; the underlying design rationale is discussed in Section 4.1.

The implemented engine integrates a Negamax search with Alpha-Beta pruning, iterative deepening, quiescence search, transposition tables, and heuristic move ordering. A dedicated time management module supports multiple search modes, including fixed depth, fixed time, node-limited, and infinite search, following the UCI protocol. In particular, a two-stage time budget with soft and hard deadlines is used to control the interaction between iterative deepening and time constraints.

Beyond this baseline, this thesis integrates and experimentally studies three further search refinements: Principal Variation Search, null-move pruning, and the reuse of search trees from previous searches through a persistent transposition table. All three are established techniques; the contribution lies in their controlled implementation and measurement within one engine, not in the algorithms themselves. The

techniques are introduced formally in Section 3 and described at the implementation level in Section 4.

The contribution of this work is threefold. First, it provides a clean and modular implementation of a classical chess engine that serves as a transparent experimental platform. Second, it extends the baseline search with Principal Variation Search, null-move pruning and transposition-based search tree reuse, and describes how these techniques are realized in practice. Third, it presents an empirical evaluation of time-dependent decision making, analyzing move stability across depths, time-to-quality curves, the depth and quality gained by each refinement, and the distribution of move quality over the position suite. On the position suite used here, the results locate the budget beyond which additional time no longer measurably improves the typical decision, and they show that the residual error is concentrated in a minority of positions that respond to additional depth far more weakly than the rest, which points to the evaluation rather than the reachable depth as the binding constraint there.

The remainder of this thesis is structured as follows. Section 2 discusses relevant prior work in computer chess and game tree search. Section 3 introduces the necessary theoretical background. Section 4 describes the architecture of the engine and the implementation of the baseline search, the Principal Variation Search, the reuse of previous search trees, the time management, and the evaluation function. Section 5 presents the experimental evaluation, and Section 6 concludes with a discussion of the results and directions for future work.



2 Related Work

The problem of efficient search in chess engines has been studied extensively over several decades [9]. Early approaches were based on the Minimax algorithm, which provides an optimal decision strategy under the assumption of perfect play [17]. However, due to the exponential growth of the game tree, Minimax quickly becomes infeasible even at moderate depths.

Alpha-Beta pruning was introduced as a major improvement over plain Minimax, allowing large parts of the search tree to be pruned without affecting the correctness of the result [7]. Subsequent work showed that the effectiveness of Alpha-Beta pruning depends crucially on the order in which moves are explored. With perfect move ordering, the effective branching factor can be reduced dramatically, enabling much deeper searches within the same computational budget [9].

Iterative deepening [8] has emerged as a standard technique to improve move ordering and to support time-controlled search. By repeatedly searching the game tree with increasing depth limits, iterative deepening ensures that a valid best move is always available while simultaneously refining the move ordering for deeper iterations. Despite the apparent redundancy caused by re-searching parts of the tree, it has been shown that the overhead is often compensated by improved pruning efficiency [9].

A detailed analysis of Alpha-Beta pruning efficiency in chess engines is presented by Nair [11]. The paper investigates how different move ordering heuristics and search enhancements affect the pruning behavior of Alpha-Beta search. In particular, it highlights the strong interaction between iterative deepening, transposition tables, and heuristic move ordering. The results indicate that improved move ordering can significantly reduce the number of explored nodes, even when additional search iterations are introduced.

Building directly on the observation that Alpha-Beta search is most effective with strong move ordering, several refinements of the basic algorithm have been proposed. Pearl introduced the Scout algorithm, which verifies promising moves with minimal-window tests before evaluating them exactly [12]. Marsland and Campbell transferred this idea to the Alpha-Beta framework in the form of Principal Variation Search [10], and Reinefeld later refined it into NegaScout, analyzing the conditions under which the additional re-searches pay off [13]. PVS is particularly effective in combination with iterative deepening and transposition tables, since both increase the probability that the first move searched is indeed the best one [9].

Transposition tables have further improved search efficiency by allowing engines to reuse previously computed results for positions reached via different move orders [18]. By storing evaluations together with depth information and bound types, transposition tables enable both direct cutoffs and enhanced move ordering. Their effectiveness is especially pronounced in combination with iterative deepening, where information from earlier iterations can be reused almost immediately [16]. Beyond single search invocations, a persistent transposition table also allows information to be carried over from one move to the next, effectively resuming previously explored search trees instead of recomputing them from scratch.

Quiescence search addresses another well-known limitation of depth-limited search, commonly referred to as the horizon effect. By extending the search in tactically unstable positions, quiescence search produces more reliable evaluations and has become a standard component of classical chess engines [9].

Beyond exact pruning, engines also employ heuristic forward pruning, which discards subtrees without proving them irrelevant. The most widely used variant is null-move pruning: if passing a turn still leaves

the position good enough to cause a cutoff, the subtree is assumed to be safe to skip. The null move as a search device is considerably older than this application of it, but the recursive form with depth reduction that engines use today was established experimentally by Goetsch and Campbell [5]. Unlike Alpha-Beta this is not value-preserving, so it trades a small risk of overlooking a line for a substantially smaller tree.

Closest to the question of this thesis is the work on how additional search actually pays off. Junghanns et al. [6] studied diminishing returns in self-play and found the winning percentage per additional ply to decrease with depth. They also report that this decrease is not visible throughout: the high error rate of chess programs hides the effect for search depths through nine plies, which is precisely the range in which the engine studied here operates. Ferreira [4] approached the same relationship from the other direction and related search depth to Elo, reporting gains per ply that shrink as depth grows.

These studies establish that the returns of additional search diminish, but they measure it through game outcomes between engines of differing depth. What they do not address is where the returns diminish *within a single engine under a clock*, at which depth its decisions stop changing, and which positions cause the remaining error. This thesis takes that perspective: instead of relating depth to playing strength, it relates the time budget to the quality of the individual decision on a fixed set of positions, and separates the positions in which additional search still changes the decision from those in which it does not.

While modern state-of-the-art engines incorporate additional techniques such as bitboards, evaluation learning, and neural networks [15], the classical search principles discussed above remain fundamental. This work builds upon these established ideas.



3 Preliminaries

This section introduces the fundamental concepts and definitions required to understand the remainder of this thesis. It establishes the formal game-theoretic model of chess, derives the associated search problem, and motivates the use of heuristic evaluation and time-constrained search. All concepts presented in this section are standard in the field of computer chess and provide a common basis for the subsequent chapters.

3.1 Chess as a Game-Theoretic Model

Chess is modeled as a deterministic, two-player, zero-sum game with perfect information, the class of games for which the classical game-tree search methods discussed below were developed [17]. At any point in time, the complete game state is fully observable to both players, and no randomness is involved in the game mechanics.

A game state consists of a board configuration together with additional state information such as the side to move, castling rights, and en-passant availability. Players alternate turns by selecting legal moves, and the game terminates in a win, loss, or draw according to the rules of chess.

More formally, let $\mathcal{T} = \{\text{pawn, knight, bishop, rook, queen, king}\}$ denote the set of piece types and $\mathcal{C} = \{\text{white, black}\}$ the set of colours. A piece is then an element of $\mathcal{F} = \mathcal{T} \times \mathcal{C}$, and the placement of pieces on the board is described by a mapping

$$b : \{1, \dots, 8\}^2 \rightarrow \mathcal{F} \cup \{\emptyset\},$$

which assigns to each square either a piece or the empty symbol $\emptyset$. A complete game state $s = (b, t, r, e)$ additionally comprises the side to move $t \in \mathcal{C}$, the castling rights r , and the en-passant target e , both of which depend on the movement history rather than on the current placement alone. The set of all legal states is denoted by $\mathcal{S}$.

The rules of chess induce a transition relation $\delta \subseteq \mathcal{S} \times \mathcal{S}$, where $(s, s') \in \delta$ if and only if s' can be reached from s by exactly one legal move. This relation defines a directed graph $G = (\mathcal{S}, \delta)$ in which each node is a legal position and each edge is a legal move. A state s is *terminal* if it has no outgoing edges, as in checkmate or stalemate, or if it is declared drawn by the rules of chess.

For decision making, the graph G is unrolled into a game tree rooted at the current position. Each level of the tree corresponds to one ply, and the number of nodes per level is governed by the branching factor of chess, which amounts to around 30 available actions per position [17].

3.2 Search Problem Formulation

Given a current position, the task of a chess engine is to select a legal move that maximizes the expected outcome under the assumption of optimal play by both sides. In the classical formulation, the possible outcomes are associated with numerical values, for example +1 for a win, 0 for a draw, and -1 for a loss.

This decision problem is commonly formulated as a game tree search problem, in which the engine explores a subset of the game tree to estimate the value of available moves. The root of the tree corresponds to the current position, while internal nodes represent positions reachable through sequences of legal moves.

With that branching factor the tree grows exponentially with depth, so evaluating all reachable positions is computationally infeasible even for moderate depths. Consequently, the search must be limited by depth, time, or other computational resources. This limitation introduces approximation: the engine can no longer guarantee optimal play, but instead aims to select the best move according to the information explored within the available budget.

The quality of a move selected by a chess engine therefore depends on two main factors: the effectiveness of the search algorithm in selecting which parts of the tree to explore, and the accuracy of the evaluation function used to estimate the value of positions at the search frontier.

3.3 Minimax, Negamax, Quiescence and Alpha-Beta Pruning

The classic algorithmic solution to game tree search in two-player zero-sum games with perfect information is the Minimax algorithm. Minimax assumes that one player aims to maximize the evaluation value of a position, while the opponent aims to minimize it. Terminal positions are assigned fixed values corresponding to win, loss, or draw, while non-terminal leaf positions are evaluated heuristically.

The algorithm explores the game tree recursively and propagates values from the leaves toward the root by alternating between maximization and minimization steps. A detailed introduction to Minimax can be found in standard textbooks on artificial intelligence in games, such as Yannakakis and Togelius [17].

Negamax Formulation In zero-sum games, the Minimax algorithm can be simplified using the Negamax formulation, a reformulation that is standard in game-tree search and treated in the surveys of the field [9, 7]. Instead of explicitly distinguishing between maximizing and minimizing players, Negamax expresses all evaluations from the perspective of the side to move. The value of a position is defined as the negation of the value obtained by the opponent after making a move.

This formulation exploits the symmetry of *two-player* zero-sum games, in which one side's gain is exactly the other's loss, so that a single score suffices if it is always read from the perspective of the side to move. It allows the search to be implemented using a single recursive procedure. Negamax is functionally equivalent to Minimax but leads to more compact and uniform code, which is why it is widely used in practical chess engines.

Quiescence Search Depth-limited game tree search combined with static evaluation functions suffers from a well-known problem referred to as the *horizon effect*. Tactical events such as captures, checks, or promotions may occur just beyond the search depth and are therefore not accounted for by the evaluation function. As a result, the engine may incorrectly assess positions as stable even though significant tactical changes are imminent.

Quiescence search is a standard technique to solve this problem. Instead of evaluating a position immediately when the nominal search depth is reached, the search is selectively extended in positions that are considered tactically unstable. Typically, only forcing moves such as captures or checking moves are explored, while quiet moves are excluded.



The goal of quiescence search is to reach a *quiescent* position, that is, a position in which no immediate tactical exchanges are available and the static evaluation is therefore more reliable. Once such a position is reached, the evaluation function is applied and the resulting score is propagated back up the search tree.

Figure 1 shows why a fixed depth limit is not merely imprecise but can be actively misleading. An exchange of pieces unfolds over several plies, and its intermediate states are wildly unbalanced: after one side has captured but before the recapture, the position looks winning by a whole piece. A search that stops exactly at that moment reports this intermediate state as the final one. The error is not small and random but large and systematic, since it always favors the side that captured last. This is also the reason a purely depth-based comparison of search algorithms is fragile without quiescence search, as one additional ply can invert the sign of the evaluation.

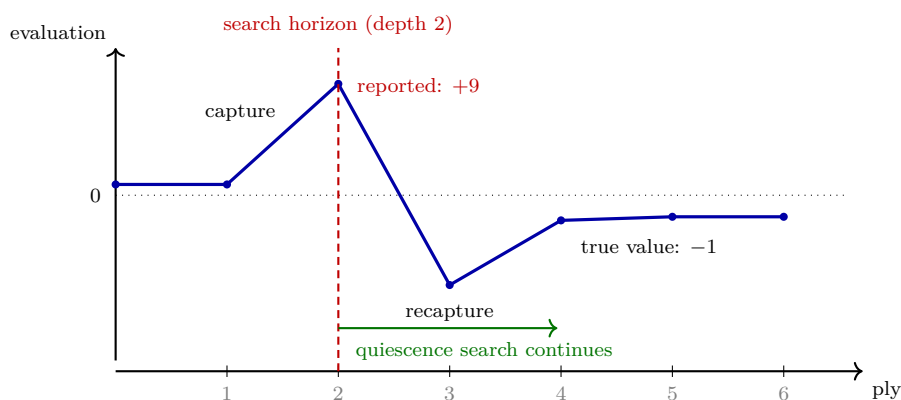


Figure 1: The horizon effect. The curve shows the static evaluation along a single line of play in which a capture is answered by a recapture. A search limited to two plies stops immediately after the capture and reports a decisive material advantage that does not exist; one ply later the material is nearly balanced again. Quiescence search resolves this by continuing beyond the nominal depth as long as captures are available, and evaluating only once the position is quiet.

In practice, quiescence search is commonly implemented using Alpha-Beta pruning with a so-called *stand-pat* evaluation [9], which represents the static evaluation of the current position before any tactical extensions are explored. If the stand-pat score already exceeds the current bounds, the node can be pruned early.

Quiescence search significantly improves evaluation stability in tactical positions and has become a standard component of classical chess engines. A comprehensive discussion of the horizon effect and quiescence search can be found in survey articles on game tree pruning [9].

Alpha-Beta Pruning Alpha-Beta pruning is an optimization technique that reduces the number of nodes that need to be explored by Minimax or Negamax without affecting the correctness of the final result. The technique was introduced by Knuth and Moore [7] and is a cornerstone of modern game tree search.

The algorithm maintains two bounds during search:

- α , the best score that the maximizing player can guarantee so far,
- β , the best score that the minimizing player can guarantee so far.

At the root of the search tree, α is initialized to $-\infty$ and β to $+\infty$. As the search progresses, these bounds are updated based on the values of explored child nodes. Whenever a node is encountered for which it can be shown that its value cannot improve the current result, the corresponding subtree is pruned and not searched further.

More concretely, during a maximizing step, the value of α is updated whenever a better move is found. If α becomes greater than or equal to β , further exploration of sibling nodes is unnecessary, as the minimizing player would avoid reaching this position. An analogous argument applies during minimizing steps.

Figure 2 illustrates this on a minimal tree. The essential point is that the pruned subtree is discarded without being looked at, and that this is nevertheless safe: the discarded branch contains the highest leaf value of the entire tree, but that value is never reachable, because the opponent decides at that node and already holds a better alternative there. Pruning therefore removes work, not information, which is what distinguishes Alpha-Beta from the heuristic pruning discussed in Section 4.2.7.

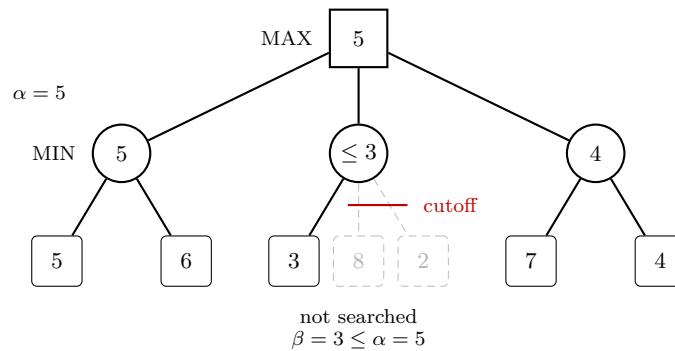


Figure 2: Alpha-Beta pruning on a minimal tree. The square root node maximizes, the circular nodes minimize. The left subtree is searched first and yields 5, so the maximizing player is guaranteed at least $\alpha = 5$. In the middle subtree the first leaf already returns 3; since the minimizing player will never choose more than 3 there, this subtree cannot beat 5 and its remaining leaves are skipped, including the 8, which is the largest value in the whole tree. The result is identical to an exhaustive search, but two of the seven leaves are never evaluated.

Effectiveness and Influencing Factors In the worst case, Alpha-Beta pruning explores the same number of nodes as plain Minimax. However, with optimal move ordering, the effective branching factor is significantly reduced, allowing the search to reach roughly twice the depth of unpruned Minimax for the same computational effort.

The effectiveness of Alpha-Beta pruning therefore depends strongly on the order in which moves are explored. Techniques such as heuristic move ordering, transposition tables, and iterative deepening are commonly employed to improve this order and thereby increase pruning efficiency. These aspects are discussed in more detail in Section 4.

Principal Variation Search Principal Variation Search (PVS) is a refinement of Alpha-Beta pruning that exploits good move ordering even further [10, 12]. The technique is based on the assumption that the first move examined at a node is usually the best one. Only this first move is searched with the full window $[\alpha, \beta]$. Every remaining move is first examined with a so-called *null window* (or zero window)



$[\alpha, \alpha + 1]$, which cannot return an exact score but can cheaply prove whether the move has the potential to improve upon α .

If the null-window search fails low, the move is confirmed to be inferior to the current best move and no further work is required. If it fails high, the assumption was wrong: the move may be better than the principal variation, and the node must be *re-searched* with the full window to obtain its exact score. PVS therefore trades occasional re-searches for substantially cheaper verification of the majority of moves. With strong move ordering, fail-highs are rare and the technique reduces the overall search effort considerably [13]. Conversely, with poor move ordering, frequent re-searches can negate the savings, which is why PVS is typically combined with iterative deepening, transposition tables, and heuristic move ordering.

For a comprehensive discussion of Alpha-Beta pruning and its variants in the context of game-playing programs, see Marsland [9].

3.4 Heuristic Evaluation

Most positions encountered during search are non-terminal and therefore cannot be assigned an exact game-theoretic value. Instead, they must be evaluated heuristically. An evaluation function assigns a numerical score to a position, indicating its desirability from the perspective of the side to move.

Classical evaluation functions combine material considerations with positional features such as piece activity, king safety, and pawn structure. These features are typically handcrafted and encode domain knowledge about the game of chess. The design of the evaluation function has a significant impact on the playing strength of the engine, particularly when search depth is limited.

Conceptually, the score returned by an evaluation function is expressed in a common unit, most often *centipawns*, where one pawn corresponds to a value of 100. Expressing heterogeneous factors, such as material imbalance and positional advantages, on a single comparable scale allows them to be combined into one number that the search can compare consistently across positions.

A refinement that is widely used in classical engines is the *tapered* evaluation. Since the relative importance of individual features changes over the course of a game, for example when kings become active in the endgame, the evaluation maintains two scores, one tuned for the midgame and one for the endgame, and interpolates between them according to a game-phase measure derived from the remaining material. The concrete evaluation used in this thesis follows exactly this scheme and is described in detail in Section 4.4.

Finally, there is an inherent tension between the richness of the evaluation and the depth of the search. A more detailed evaluation captures more chess knowledge per node but is more expensive to compute, which reduces the number of nodes that can be searched within a fixed time budget.

3.5 Time-Constrained Search

In practical settings, chess engines are required to operate under strict time constraints. Rather than searching to a fixed depth, the engine must dynamically adapt its search behavior to the available time budget.

Iterative Deepening Iterative deepening addresses this requirement by turning a single fixed-depth search into a sequence of searches with increasing depth limit $d = 1, 2, 3, \dots$, each starting again from the root [8]. After every completed iteration the engine holds a fully valid best move for that depth, so the search can be interrupted at essentially any point and still return a well-founded decision. This *anytime* property is precisely what makes the technique well suited to time-controlled play, where the exact amount of available time is not known in advance.

At first glance, repeatedly re-searching the shallow part of the tree appears wasteful. In practice the overhead is small: because the number of nodes grows exponentially with depth, the cost of the final iteration dominates the sum of all previous ones, so re-searching depths 1 to $d - 1$ adds only a constant factor to the cost of the depth- d search. As discussed next, this overhead is typically more than recovered through the improved move ordering that earlier iterations provide.

Figure 3 makes the two consequences of this exponential growth visible at once. Because each iteration costs roughly three times its predecessor, everything before the last one is nearly free, which is what makes the repetition affordable. The same fact, however, means the engine faces an all-or-nothing decision at every iteration boundary: the next iteration will consume more time than the entire search so far, and if the clock runs out half-way through it, that work is discarded entirely. Deciding whether to start another iteration is therefore not a minor scheduling detail but the central question of time management, and it is addressed in Section 4.3.

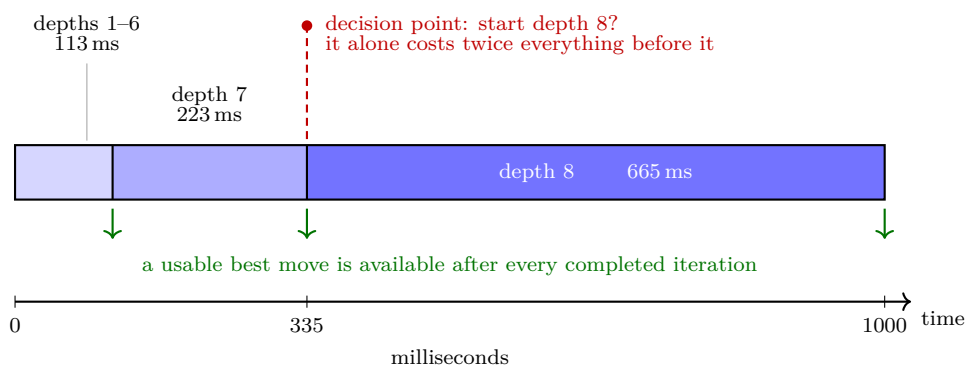


Figure 3: Iterative deepening under a one-second budget, with the width of each segment proportional to the time that iteration consumes. The proportions follow from the effective branching factor of about 3 measured for this engine in Section 5.6. The first six iterations together account for roughly a tenth of the budget, so the repeated re-searching is cheap; the last iteration alone accounts for two thirds. Every completed iteration leaves a valid best move behind, which is what allows the search to be stopped at any point.

Interaction with Pruning In addition to providing robustness under time constraints, iterative deepening also improves the effectiveness of Alpha-Beta pruning. Information gathered in earlier iterations, such as good move orderings, can be reused in deeper searches, leading to a significant reduction in the number of explored nodes.

Search Tree Reuse Across Searches The same principle extends beyond a single search invocation. After the engine has played a move and the opponent has responded, the new search root lies within the tree that was explored during the previous search. If the results of that search are retained, for example in a transposition table, the engine can *resume* the old search tree: previously computed scores yield immediate



cutoffs, and previously identified best moves improve the move ordering from the very first iteration. Conceptually, the engine no longer starts each search from scratch but continues the computation it has already begun in earlier moves of the game.

The interaction between heuristic evaluation, Alpha-Beta pruning, and time-constrained iterative deepening forms the foundation of modern chess engines and is a central aspect of this work.



4 Algorithm & Implementation

This section presents the algorithms used by the engine and how they were implemented. It first introduces the overall engine architecture and the baseline search, consisting of Negamax with Alpha-Beta pruning, iterative deepening, quiescence search, transposition tables, and heuristic move ordering. Building on this baseline, the three search refinements evaluated in this thesis are presented: Principal Variation Search, null-move pruning, and the reuse of search trees from previous searches. The section concludes with the time management scheme and the static evaluation function.

4.1 Engine Architecture

The implemented chess engine follows a layered architecture whose structural design decisions and component interactions are the focus of this subsection; the rationale behind these decisions is summarized at the end.

— 4.1.1 Overall Design

The engine follows a layered architecture that separates game representation, move generation, search, evaluation, and search control. Each component has a clearly defined responsibility and communicates with other modules through narrow and explicit interfaces.

Figure 4 provides a high-level overview of the engine structure and the interaction between the individual modules.

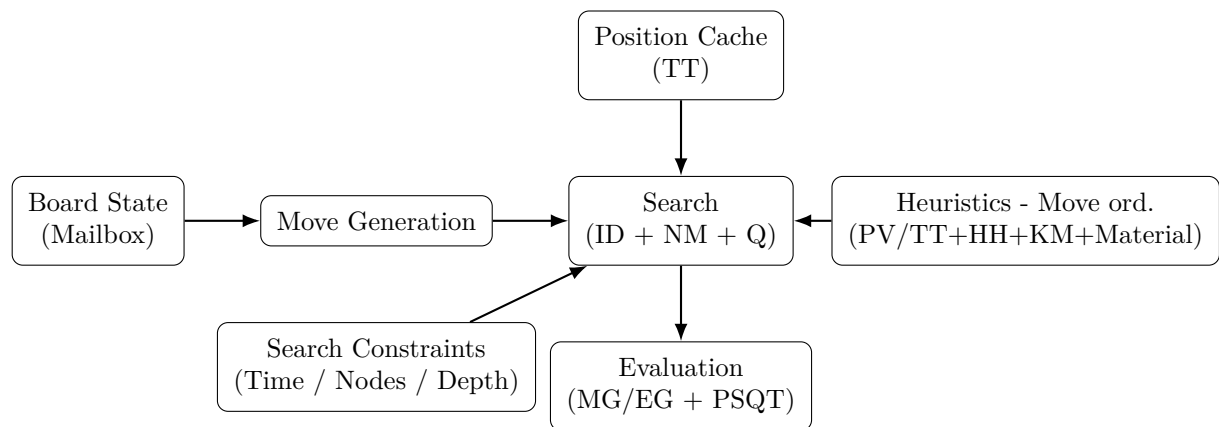


Figure 4: High-level architecture of the implemented chess engine.

At a conceptual level, the engine consists of the representation of the game state, the generation and application of moves, the search procedure, the static evaluation, and a unified constraint mechanism that controls search termination. This separation ensures that individual components can be modified or replaced without affecting the rest of the system, which is particularly important for experimental analysis.

— 4.1.2 Game State and Move Handling

Board Representation The board is represented using a classical mailbox model consisting of 64 squares. Each square explicitly stores the occupying piece or indicates that the square is empty. In addition to the piece placement, the board state contains information about the side to move, castling rights, king squares and en-passant availability.

Modern state-of-the-art engines typically rely on bitboard-based representations for efficient move generation and evaluation. However, mailbox representations remain a common choice in educational and experimental engines due to their simplicity and transparency [1]. Given the focus of this work on algorithmic behavior rather than raw performance, the mailbox model provides a clear and easily analyzable foundation.

It is worth being explicit about what this decision costs, since it is the single most consequential one in the architecture. The engine sustains roughly 1.7 million nodes per second and reaches an average depth of 8.2 plies at a one-second budget (Section 5.6). A bitboard implementation would plausibly search several times as many nodes per second, but because depth grows only logarithmically with search effort, even a fivefold faster move generator would buy no more than about 1.5 additional plies at the same budget. The representation therefore shifts the operating point of the measurements without changing their structure: every result in this thesis is a statement about how decision quality behaves *as a function of* depth and budget, and such a statement is displaced along the axis by a faster engine, not invalidated. The same reasoning is the reason absolute playing strength is treated throughout as a sanity check rather than as a result.

Move Representation Moves are represented as explicit objects containing all information required to apply and undo them. This includes the source and destination squares, the moving piece, captured pieces, promotion information, and special move flags for castling, promotion and en-passant captures.

This design enables a fully reversible make-unmake mechanism, which is essential for recursive search algorithms. By storing all relevant information directly in the move object, the engine avoids recomputation when restoring previous board states, thereby simplifying the search implementation and reducing potential sources of errors.

Move Generation Move generation follows a direct and exhaustive approach. For each piece on the board, all potential target squares are examined according to the rules of chess. No precomputed attack tables or bitboard-based optimizations are used.

Illegal moves, such as those that leave the king in check, are filtered during move execution rather than during generation. This design keeps the move generator independent of check detection logic and makes its behavior easier to reason about. Although this approach is less optimized than state-of-the-art alternatives, it provides a robust and well-understood baseline for search experimentation.

— 4.1.3 Position Hashing and Transposition Tables

Zobrist Hashing To uniquely identify board positions, the engine employs Zobrist hashing. Each combination of square and piece type is associated with a random 64-bit value, and the hash of a position



is computed by XOR-combining the values corresponding to the pieces currently on the board. Additional state information, such as the side to move, castling rights, and en-passant availability, is also incorporated into the hash.

Zobrist hashing is commonly used in game tree search because it allows board positions to be mapped efficiently to fixed-size keys. The technique supports incremental updates during move making, where only the keys of the affected squares are combined into the existing hash. This engine instead recomputes the hash from the board state whenever a move is applied, which keeps the hash independent of the move-making code: castling, en-passant and promotion each modify several squares at once, and an incremental update has to mirror every one of these cases correctly, whereas a full recomputation cannot fall out of sync with the board. Undoing a move needs no recomputation at all, since the hash is stored alongside the board settings in the move history and restored from there. The technique itself was originally introduced by Zobrist [18] and has since become a standard component of modern chess engines.

Note that the en-passant target square is part of the hashed state, so a position in which a pawn has just advanced two squares hashes differently from the otherwise identical position without that pawn move. Setting the target square after every double step would therefore split positions that are in fact equivalent, since en passant is only relevant when an enemy pawn is actually positioned to capture. The engine consequently only records the target square when a capturing pawn stands next to the advanced pawn, which raises the transposition table hit rate from 0.111 to 0.118 at a one-second budget over the benchmark suite.

Transposition Table Usage The computed hash values are used as keys in a transposition table, which stores previously evaluated positions. Each entry contains the evaluation score, the search depth at which the position was evaluated, a bound type (exact, lower bound, or upper bound), and the best move found.

The transposition table serves two closely related purposes. First, it reduces search redundancy by reusing results for positions that can be reached via different move orders, which is common in chess. Without such a mechanism, identical positions would be evaluated repeatedly, leading to a significant increase in search effort. Second, transposition table entries improve move ordering by prioritizing moves that proved effective in previous searches. A simple aging mechanism ensures that outdated entries are gradually replaced, preventing the table from being dominated by stale information. Since the table persists across search invocations, it additionally serves as the mechanism by which the engine resumes previously explored search trees, as described in Section 4.2.9. For practical implementations and further discussion, see for example Warnock [16].

— 4.1.4 Search Control and Integration

Unified Search Constraints All termination conditions for a search invocation are encapsulated in a single constraint structure. This includes depth limits, node limits, time budgets, and special modes such as infinite search or pondering.

Encapsulating all stopping criteria in a unified constraint object avoids scattered stop checks throughout the search code and ensures consistent behavior across different search modes. This design is particularly important for experimental analysis, as it guarantees that differences in search behavior are caused by the chosen constraints rather than by implementation artifacts.

Engine Integration The engine core is coordinated by a game management component that maintains the current position, applies and reverts moves, and invokes the search procedure. This component acts as a bridge between the external UCI interface and the internal search and evaluation modules, ensuring a clean separation between protocol handling and algorithmic logic.

— 4.1.5 Universal Chess Interface (UCI)

Externally, the engine is controlled through the *Universal Chess Interface* (UCI), the most widely used communication protocol in modern computer chess. UCI specifies a text-based command interface that allows external tools to set up positions, initiate searches, and receive search results; search constraints such as time limits, depth limits, node limits, and pondering are communicated via the `go` command.

The protocol itself is independent of the engine’s internal implementation and therefore separates external control from internal search logic. This separation is reflected directly in the design described above: incoming UCI parameters are first mapped to the abstract **SearchConstraints** representation (Section 4.3) before being enforced during search. All experiments reported in this work are conducted through UCI-compatible interfaces and tournament frameworks, which makes UCI the practical context in which time-constrained search and iterative deepening are evaluated.

— 4.1.6 Design Rationale

The architectural decisions made in this engine deliberately prioritize clarity, correctness, and experimental flexibility over maximum playing strength. Many state-of-the-art engines employ highly optimized data structures and aggressive micro-optimizations, which can obscure the underlying algorithmic behavior.

By choosing a modular and explicit design, this engine allows the effects of search algorithms, heuristics, and time management strategies to be studied in isolation. This aligns with the primary goal of this work, which is to analyze time-dependent decision making rather than to compete directly with top-performing chess engines.

4.2 Search Algorithms

The underlying algorithms (Negamax, Alpha-Beta pruning, and quiescence search) were introduced in Section 3. This subsection turns to their concrete realization in code and to how the individual mechanisms interact in practice. Particular attention is paid to the two refinements of this thesis: the move loop is organized as a Principal Variation Search (Section 4.2.5), and previous search trees are resumed through the persistent transposition table (Section 4.2.9).

— 4.2.1 Search Entry Point and Modes

The main entry point is `ChessBot::think()`, which receives the board together with a **SearchConstraints** object. The constraints are copied intentionally and stored inside the engine instance for the duration of the search. Before searching, the engine initializes a clean state via `reset_search_state()`, which



resets node counters, selective depth tracking, stop state, and heuristic tables (killer and history), and also advances the transposition table generation via `tt.new_search()`.

Depending on `config.mode_`, the engine selects one of the following execution strategies:

- **FixedDepth:** the time budget is explicitly disabled by leaving `budget_` empty, which prevents time-based termination inside `hard_stop()`. The engine performs a single root search with the requested depth and returns the result.
- **NodeLimit / Infinite:** time-based termination is disabled and the engine runs iterative deepening until an external stop condition (or the node limit) is hit.
- **Normal / FixedTime (time-managed):** the `TimeManager` initializes a soft and hard deadline from the constraints, and iterative deepening runs under these deadlines.

This structure keeps the actual search logic independent of UCI parsing and ensures that all termination behavior is expressed through the constraint object and the common stop checks.

— 4.2.2 Termination and Safety Checks

All immediate termination decisions are centralized in `ChessBot::hard_stop()`, which is called frequently inside the search (including quiescence) and combines an external stop flag, the hard time deadline, and an optional node budget. The exact semantics of these criteria, and the distinction between the hard deadline and the *soft* deadline that is only consulted between completed iterations of iterative deepening, are described in Section 4.3.

The stop flag and the node budget are checked at every node, the clock only every 1024 nodes. Reading a monotonic clock costs considerably more than the comparison it guards, and at the node rates the engine reaches, the resulting imprecision stays below a millisecond.

As an additional robustness measure, the engine verifies the final move before returning it. If the selected move is not legal (checked via `is_legal_move()`), the engine falls back to a legal move obtained from `get_legal_fallback_move()`.

— 4.2.3 Iterative Deepening Loop

Time-managed searches run in `ChessBot::iterative_deepening(Board&)`. The engine starts with a legal fallback move and then performs repeated searches with increasing depth $d = 1, 2, 3, \dots$.

For each iteration, a local copy of the current best move is passed into `root_search()`. If the search completes successfully, the resulting move becomes the new best move and is reported via the UCI `info` output (`print_info`). If an iteration is aborted (e.g., due to a hard stop), the loop terminates and the engine returns the best move from the last completed iteration.

Hard stops are enforced immediately inside the search, while soft stops only take effect at iteration boundaries (cf. Section 4.3).

— 4.2.4 Root Search and Core Negamax

The helper `root_search(Board&, depth, Move&)` is a thin wrapper that invokes Negamax with the initial window $[-\infty, +\infty]$ and ply 0. The best move is returned through the referenced move parameter, which avoids additional return-value plumbing and keeps the recursive signature compact.

The core search routine `ChessBot::negamax(...)` implements a standard depth-limited Negamax with Alpha-Beta bounds:

- If `depth <= 0`, the search transitions into quiescence search.
- At every node, `hard_stop()` may terminate the search early.
- The routine maintains and updates α during move exploration; a cutoff occurs when $\alpha \geq \beta$.

To support correct transposition table storage, the implementation keeps the original alpha value in `ORIGINAL_ALPHA` and uses it afterwards to classify the result: as `EXACT` if the score lies inside the window, as `LOWER_BOUND` if the node failed high, and otherwise as `UPPER_BOUND`.

Legal Move Handling Moves are generated as pseudo-legal moves and filtered by attempting to apply them with `board.make_move(move)`. If `make_move` fails (e.g., because the move leaves the king in check), the move is discarded. This design keeps move generation simpler and delegates legality to the `make-unmake` layer, which also guarantees that the search only recurses into legal positions.

Terminal Positions If no legal move exists, the implementation distinguishes:

- **checkmate:** if the side to move is in check, return a mate score of the form $-(MATE - ply)$,
- **stalemate:** otherwise return 0.

Using the current ply in the mate score ensures that faster mates are preferred and slower mates are avoided.

— 4.2.5 Principal Variation Search

The move loop inside `negamax` implements the Principal Variation Search scheme introduced in Section 3. Instead of searching every move with the full window $[\alpha, \beta]$, the engine searches only the leading moves with the full window and verifies all remaining moves with a null-window search $[\alpha, \alpha + 1]$ (realized as $[-\alpha - 1, -\alpha]$ in the Negamax formulation). Only if the null-window search indicates that a move can beat α without already exceeding β is the move re-searched with the full window to obtain its exact score.

The behavior is controlled by two tunable parameters bundled in a `PvsConfig` structure:

- **min_depth** (default 2): below this remaining depth the scout search is skipped entirely and all moves are searched with the full window. At shallow nodes the subtrees are so small that the potential re-search overhead would outweigh the savings of the null-window search.



- `scout_after_move` (default 1): the first N legal moves of a node are always searched with the full window. Move ordering is least reliable for the very first moves, so the engine only switches to the cheap null-window scout once the presumed principal variation has been established.

Both parameters are exposed as UCI options (`PvsMinDepth` and `PvsScoutAfterMove`), which allows the scouting behavior to be tuned and evaluated experimentally without recompiling the engine. The overall structure of the move loop is summarized in Algorithm 1.

Algorithm 1: Principal Variation Search inside the Negamax move loop.

Input: Position P , depth d , bounds α, β , legal move counter n
Output: Score of the searched move

```

1 if  $n < \text{scout\_after\_move}$  or  $d < \text{min\_depth}$  then
2   |  $\text{score} \leftarrow -\text{NEGAMAX}(P, d-1, -\beta, -\alpha)$  // full window
3 else
4   |  $\text{score} \leftarrow -\text{NEGAMAX}(P, d-1, -\alpha-1, -\alpha)$  // null-window scout
5   | if  $\alpha < \text{score} < \beta$  then
6   |   |  $\text{score} \leftarrow -\text{NEGAMAX}(P, d-1, -\beta, -\alpha)$  // re-search
7 return  $\text{score}$ 
```

A subtle but important implementation detail is the interaction with aborted searches: if a null-window scout is aborted by a hard stop, its score is not trusted and no re-search is triggered; instead, the abort is propagated upward so that the iterative deepening loop can discard the incomplete iteration.

— 4.2.6 Quiescence Search

Quiescence search is implemented in `ChessBot::quiescence(...)` and is entered whenever the regular search reaches depth 0. The function evaluates the current position via `eval::evaluate(board)` and then selectively extends the search by exploring only tactical moves (captures and en-passant). Non-capturing moves are skipped explicitly. The captures are filtered out of the generated list before the list is ordered, so the ordering only has to run over the handful of moves that are actually searched instead of over all of them.

The implementation uses the standard *stand-pat* scheme:

- if the static score already exceeds β , the node fails high,
- otherwise α is updated using the static score and improved through capture exploration.

As with the main search, `hard_stop()` is checked inside quiescence to enforce hard termination constraints consistently.

— 4.2.7 Null-Move Pruning

In addition to the exact Alpha-Beta framework, the engine applies null-move pruning as a forward-pruning enhancement of the baseline search. The underlying observation is that in most positions having the right to move is an advantage: if a side is allowed to skip its turn (a *null move*) and the resulting position,

searched to a reduced depth, is still good enough to produce a beta cutoff, then the position is almost certainly too strong for the opponent and the whole subtree can be pruned without a full-depth search. The recursive form with depth reduction implemented here follows Goetsch and Campbell [5] and is a standard component of classical engines.

Figure 5 contrasts the two searches involved. The decisive property is that the reduced search answers a deliberately easier question than the one the node actually poses. It grants the opponent a free move and still finds the position too good, which is strong evidence that the node would also survive a proper search. Unlike Alpha-Beta this is evidence rather than proof, and the figure also indicates where it breaks down: in zugzwang positions, having to move is a disadvantage, so passing improves rather than worsens the position and the inference is invalid. This is the reason for the guard conditions listed below.

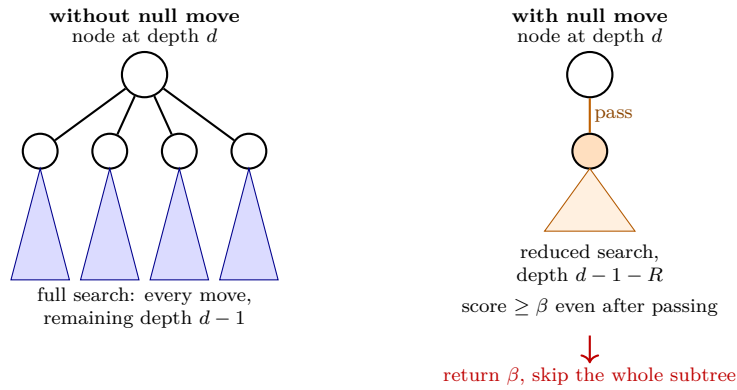


Figure 5: Null-move pruning. Instead of searching every move at full depth (left), the engine first lets the side to move pass and searches the resulting position to a reduced depth (right). If the position is still good enough to exceed β despite this handicap, the node is assumed to be beyond the opponent’s reach and the subtree is skipped. The saving is large because the probe is both shallower and single-branched, but the inference is heuristic: it fails in zugzwang, where passing would be an improvement rather than a concession.

The pruning is performed inside `negamax`, after the transposition table probe but before the moves are generated and ordered. The engine plays a null move via `board.make_null_move()`, searches the resulting position to a reduced depth $d-1-R$ with a null window around β (realized as $[-\beta, -\beta+1]$ in the Negamax formulation), and restores the position with `board.pop_null_move()`. If the returned score satisfies $-\text{score} \geq \beta$, the node fails high and β is returned directly. The number of such cutoffs is tracked in the `null_cutoffs` counter for the experimental instrumentation.

Because null-move pruning is a heuristic rather than an exact technique, it is guarded by several conditions. The reduced search is only attempted when all of the following hold:

- pruning is enabled and this node was not already reached through a null move (`can_null`); two consecutive passes are never searched, as they correspond to no meaningful line of play,
- the node is not the root (`ply > 0`), since the root must always return a real best move,
- the remaining depth is at least `min_depth`; below this the reduced search would be too shallow to justify the risk,
- β is not already a mate score ($|\beta| < \text{kMate} - \text{kMateWindow}$), because a fabricated move would distort



mate distances,

- the side to move is not in check, since passing while in check would leave the king en prise,
- the side to move still has non-pawn material (`has_non_pawn_material`); in pawn-only endgames zugzwang makes passing frequently favourable, which would invalidate the pruning assumption.

As in the rest of the search, an aborted reduced search is not trusted: if the null-move search is terminated by a hard stop, the abort is propagated upward and no cutoff is derived from the incomplete result. The behaviour is controlled by a `NullMoveConfig` structure exposing whether the technique is enabled, the minimum remaining depth (`min_depth`, default 3), and the depth reduction (`reduction`, default 2). All three are exposed as the UCI options `NullMove`, `NullMoveMinDepth`, and `NullMoveReduction`, so the pruning can be toggled and tuned without recompiling.

— 4.2.8 Transposition Table Integration

The motivation for the table is that the game tree is not, strictly speaking, a tree. Different sequences of moves frequently lead to the identical position, so the same node appears many times over at the same depth. A plain recursive search has no way of noticing this and re-solves each occurrence from scratch, which is pure duplicated work. Figure 6 shows the effect in its smallest form. Storing results under a hash of the position turns the traversal from a tree into a directed acyclic graph and lets the second occurrence be answered from memory. Since the number of such collisions grows with depth, this is one of the few techniques whose benefit increases rather than decreases as the search gets deeper, which is also why the same table can be carried across moves (Section 4.2.9).

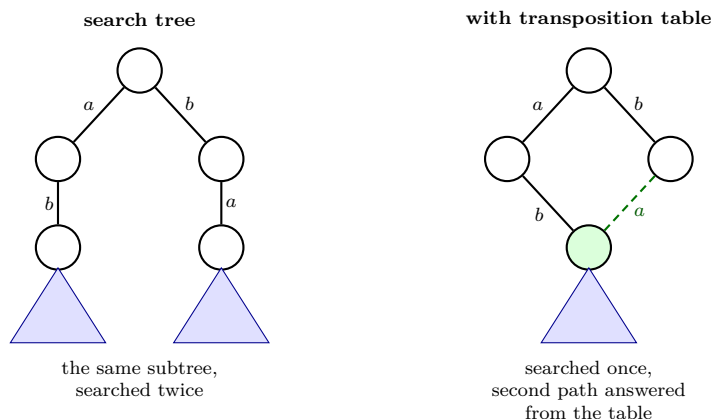


Figure 6: Transpositions. Playing move *a* and then *b* leads to exactly the same position as playing *b* and then *a*, so a plain recursive search explores the identical subtree twice (left). A transposition table stores the result of the first visit under a hash of the position, so the second path finds it already solved and returns immediately (right). Real search trees contain vastly more such convergences than this minimal example suggests.

The search consults the transposition table early in `negamax` via `tt.probe(key, depth, alpha, beta, ply, ...)`. A successful probe may:

- return an **EXACT** score,

- produce a safe cutoff using stored `LOWER_BOUND` or `UPPER_BOUND` information.

In addition, the table provides a stored best move (`tt_move`) which is used for move ordering. Since transposition table entries may be stale or collide, the implementation verifies the move's legality before using it; illegal TT moves are discarded.

After searching a node, the result is stored via `tt.store(...)` together with the depth, bound type, and the best move found at that node. Mate scores are encoded in a TT-safe form using the ply (via `to_tt_score / from_tt_score`) so that mate distances remain consistent across different search depths.

— 4.2.9 Search Tree Reuse Across Searches

The transposition table is not only used within a single search invocation but also serves as the engine's mechanism for resuming previous search trees. The table is a member of the engine instance and is deliberately *not* cleared between searches. When a new search starts, `reset_search_state()` only calls `tt.new_search()`, which advances an internal generation counter; all stored entries remain accessible. A full `tt.clear()` is performed only when the UCI command `ucinewgame` signals that an entirely new game begins.

This design directly implements the idea of continuing an earlier computation. After the engine has played a move and the opponent has answered, the new root position lies inside the tree that was explored during the previous search. Probing the persistent table therefore immediately recovers large parts of the earlier work:

- **Score reuse:** entries stored with sufficient depth yield exact scores or safe bound cutoffs without re-searching the corresponding subtrees.
- **Move ordering reuse:** stored best moves from the previous search guide the move ordering of the new search from the very first iteration, which is exactly the situation in which Principal Variation Search and Alpha-Beta pruning are most effective.

The generation counter implements a soft aging scheme rather than hard invalidation. The replacement policy prefers overwriting entries from older generations and entries with lower depth, so stale information gradually disappears as the game progresses while still being available as long as it is not displaced. In combination with iterative deepening, this means consecutive searches behave less like independent computations and more like one continuous search that is deepened and re-rooted as the game advances.

Keeping the table alive across searches has one consequence that a table cleared per search does not have: a stored move outlives the position it was generated for. Since the table is indexed by a truncated key, two different positions can map to the same entry, and the move handed back then describes pieces that are no longer where it expects them. Such a move is not merely bad, it is not playable at all, and applying it would corrupt the board rather than just cost search time. Every move taken from the table is therefore validated against the current position before it is used as an answer: it has to be one the move generator produces here, and it must not leave the own king in check. Moves that come straight from the generator need no such check, they are pseudo-legal by construction, so the validation is confined to the few places where a move of foreign origin enters the search.



— 4.2.10 Move Ordering and Heuristics

Move ordering is implemented in `search::heuristics::order_moves`. The ordering score combines several standard heuristics, all of which are surveyed by Marsland [9]:

- **TT/PV move first:** if a transposition table move exists, it receives a large bonus and is sorted to the front.
- **Captures before quiet moves:** captures are prioritized using a MVV-LVA-inspired score (most valuable victim / least valuable attacker), with a special case for en-passant.
- **Killer moves:** for each ply, the engine tracks up to two quiet moves that previously caused beta cutoffs and boosts them in ordering.
- **History heuristic** [14]: a 3D table indexed by side, from-square, and to-square accumulates bonuses (scaled by depth squared) for quiet moves that caused cutoffs, and the accumulated score is added during ordering.

Killer and history updates are performed only on beta cutoffs and only for quiet moves. This avoids polluting these tables with captures and ensures that the heuristics reflect moves that are generally useful for pruning rather than tactical one-offs.

— 4.2.11 Debugging and Instrumentation

The search provides UCI-compliant reporting via `print_info`, including depth, selective depth, time, nodes, NPS, and either centipawn score or mate distance (converted from the internal mate representation). Additionally, a configurable debug facility can emit search health statistics, transposition table statistics, root move ordering, and principal variation output. This instrumentation supports the experimental goal of this thesis by making search behavior observable and reproducible.

4.3 Time Management and Constraints

In practical chess engines, search is not only limited by depth but often by a set of external constraints such as time, node budgets, or explicit stop signals. In this engine, all stopping conditions for a single search invocation are bundled in an immutable constraint object, which is initialized once before the search starts and treated as read-only during the search.

A key design goal is that the search code itself does not depend on UCI parsing details. Instead, the UCI layer produces a fully specified `SearchConstraints` instance, and the search only consults this object (and a stop flag) to decide when to terminate.

— 4.3.1 Unified Constraint Model

The engine uses a dedicated `SearchConstraints` structure to describe how a search is controlled and when it should terminate. This structure unifies all search modes under a single interface and avoids scattered termination logic across the search code.

A constraint instance contains:

- a **search mode** determining which criteria are active,
- an optional **fixed move time** (`movetime_ms_`),
- an optional **fixed depth** (`depth_`),
- an optional **node limit** (`nodes_`),
- and a **time budget** object (`budget_`) used for time-managed searches.

To simplify checks in the search, convenience predicates such as `has_fixed_depth()`, `has_node_limit()`, and `has_movetime()` are provided.

— 4.3.2 Search Modes

The active search behavior is selected via `SearchType`. The engine supports the following modes:

- **Normal:** Tournament-style time management with iterative deepening (default UCI behavior). Time allocation is derived from the clock settings and stored in `budget_`.
- **FixedTime:** Search for a fixed amount of time specified by `movetime_ms_` (UCI go `movetime`).
- **FixedDepth:** Search to a fixed depth only, determined by `depth_`, without relying on time control.
- **NodeLimit:** Search until a fixed number of nodes specified by `nodes_` has been explored.
- **Infinite:** Search indefinitely until an external stop command is received (UCI go `infinite`).
- **Ponder:** Search during the opponent's time. The engine continues searching until it is either stopped or the predicted opponent move is played, at which point the search can be continued or restarted.

This mode-based design enables a clean separation between the UCI interface (which interprets the `go` command) and the search itself (which only consults the active constraints).

— 4.3.3 Time Budgets: Structure and Arming

Time-based searches (Normal / FixedTime) rely on `TimeControl` stored in `SearchConstraints::budget_`. The budget distinguishes a *soft* and a *hard* limit:

- **Soft deadline:** preferred stopping point, checked only at safe boundaries (between completed iterations of iterative deepening).
- **Hard deadline:** strict upper bound that must not be exceeded, checked frequently throughout the search (including quiescence).



Unarmed Budgets for Non-time Modes A crucial implementation detail is that `budget_` is explicitly *disarmed* for non-time-based modes. Concretely, deadlines are set to zero, so `soft_time_up(...)` and `hard_time_up(...)` always return false. This ensures that depth-limited and node-limited search do not depend on any timing state.

Arming the Budget Before Search Immediately before a time-managed search begins, the engine calls `TimeManager::init_search`. This function captures the current monotonic time (`now_ms()`) as the start timestamp and derives absolute deadlines:

```
soft_deadline_ms_ = start_ms_ + soft_ms_,    hard_deadline_ms_ = start_ms_ + hard_ms_.
```

From this point onward, the budget object is treated as read-only.

— 4.3.4 Termination Conditions in the Search

The engine enforces termination using a combination of (i) an external stop flag, (ii) the hard deadline, and (iii) an optional node limit. These checks are centralized in `ChessBot::hard_stop()`, which is called frequently inside the main search and quiescence.

Hard Stop Checks The stop check follows the order shown in Algorithm 2: first an external flag, then the hard time deadline, then the node limit. If any condition is met, the search aborts immediately and returns a special “aborted” result to the caller.

Algorithm 2: Immediate termination check (`hard_stop()`).

Input: Current time *now*
Output: `true` iff the search must terminate immediately

```
1 if stop_requested is set then
2   return true
3 if budget_.hard_time_up(now) then
4   return true
5 if has_node_limit() and nodes >= nodes_ then
6   return true
7 return false
```

Soft Stop in Iterative Deepening In contrast to the hard deadline, the *soft* deadline is checked only after a complete iteration has finished. This ensures that the engine does not stop mid-iteration, which would otherwise risk returning an unstable or partially explored root result. The structure is summarized in Algorithm 3.

— 4.3.5 Summary

The engine models time control and other stopping criteria using a single, immutable `SearchConstraints` object. By activating different termination rules depending on `SearchType` and by explicitly arming or

Algorithm 3: Soft stopping policy at iteration boundaries (iterative deepening).

Input: Board position P **Output:** Best move found within the budget

```
1  $bestMove \leftarrow$  legal fallback move
2 for  $d \leftarrow 1, 2, 3, \dots$  do
3    $(score, aborted) \leftarrow$  depth-limited root search at depth  $d$ 
4   if  $aborted$  then
5     break                                     // do not trust partial iteration
6    $bestMove \leftarrow$  move from completed iteration
7   if  $budget\_soft\_time\_up(now)$  then
8     break                                     // stop cleanly after a full iteration
9 return  $bestMove$ 
```

disarming time budgets, the implementation remains robust and easy to analyze.

Most importantly, the implementation distinguishes between *hard* stopping conditions, which are enforced immediately inside the recursive search, and a *soft* stopping condition, which terminates the search cleanly between iterations of iterative deepening. This separation is essential for making time-constrained iterative deepening comparable to fixed-depth search in later experiments, since it ensures that the engine always returns the best move from the last fully completed depth.

4.4 Evaluation Function

Since the search explores only a limited portion of the game tree, a heuristic evaluation is required to estimate the quality of non-terminal positions. The implemented evaluation follows a classic *tapered* approach with separate midgame (MG) and endgame (EG) scores, piece-square tables (PSQT), a phase-based interpolation scheme, and a tempo bonus. Its material values and tables are adapted from the widely used PeSTO tables [3].

— 4.4.1 Material Model

The evaluation assigns a material value to each piece type. Material is counted for both sides and is added to the positional contribution of the corresponding piece-square table. Conceptually, for a piece on square s , the score is:

$$\text{score}(p, s) = \text{material}(p) + \text{psqt}(p, s).$$

In the implementation, material values are used in two variants: a midgame value and an endgame value. This is consistent with the idea that the relative importance of pieces changes over the course of a game (e.g., kings become more active in endgames).

— 4.4.2 Piece-Square Tables (PSQT)

Positional preferences are encoded using piece-square tables. For each piece type, the engine stores a 64-entry table for midgame and endgame, assigning a positional bonus (or penalty) to each square. This



model captures common chess principles such as centralization, piece activity, and king safety without explicit feature engineering.

The implementation uses a single table orientation and mirrors squares for the white pieces by applying a rank flip. Concretely, for a board index i , white pieces use index $i \oplus 56$, while black pieces use i directly. This ensures that both colours share the same PSQT semantics relative to their respective perspective.

— 4.4.3 Midgame and Endgame Separation

Instead of a single score, the evaluation computes two scores:

- MG : midgame score (material + PSQT using MG tables),
- EG : endgame score (material + PSQT using EG tables).

The engine accumulates these values separately for both sides. The final score is then expressed from the perspective of the *side to move*. This is implemented by subtracting the opponent's accumulated values from the values of the player whose turn it is.

— 4.4.4 Game Phase Interpolation

To smoothly transition between midgame and endgame behavior, the engine uses a tapered evaluation. A game-phase value is computed as the sum of per-piece phase contributions. The resulting phase is clamped to a maximum of 24, yielding:

$$mgPhase = \min(24, \text{phase}(P)), \quad egPhase = 24 - mgPhase.$$

The final evaluation is obtained by linear interpolation:

$$Eval(P) = \frac{MG(P) \cdot mgPhase + EG(P) \cdot egPhase}{24}.$$

This approach is commonly used in classical engines, as it avoids abrupt evaluation changes and provides a simple, interpretable blending scheme. Figure 7 shows the two weights over the course of a game: as material leaves the board, the endgame table gradually takes over from the midgame table, so that a king which was penalized for leaving its shelter early on is rewarded for centralizing later, without either table ever switching on abruptly.

— 4.4.5 Tempo Bonus

A constant tempo bonus is added to the final score to reflect the advantage of having the next move. In the implementation, the returned evaluation is:

$$Eval_{\text{final}}(P) = Eval(P) + 20.$$

Since the engine evaluates positions from the perspective of the side to move, this constant shifts the score slightly in favor of the player whose turn it is, encouraging initiative and reducing symmetry artifacts.

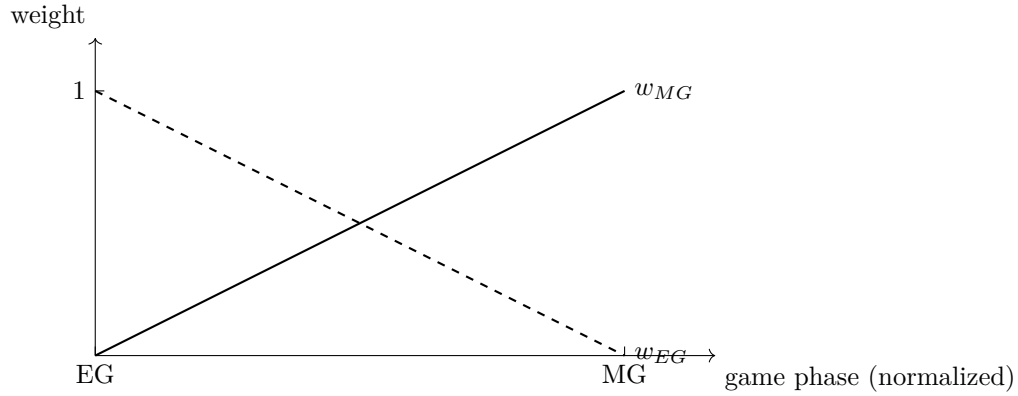


Figure 7: Tapered evaluation: midgame and endgame weights vary linearly with the game phase.

4.4.6 Algorithmic Summary

Algorithm 4 combines the parts described above into the procedure the engine actually executes. A single pass over the 64 squares accumulates the midgame and endgame scores for both sides together with the game phase, so that material, piece-square values and phase are collected in one traversal. The two scores are then formed from the perspective of the side to move, interpolated according to the phase, and the tempo bonus is added.

Algorithm 4: Static evaluation with MG/EG separation, tapered interpolation, and tempo bonus.

Input: Board position P

Output: Static evaluation score from the perspective of the side to move

```

1 Initialize  $mg[2] \leftarrow 0$ ,  $eg[2] \leftarrow 0$ ,  $phase \leftarrow 0$ 
2 for  $i \leftarrow 0$  to 63 do
3    $piece \leftarrow P[i]$ 
4    $phase \leftarrow phase + phaseValue(piece)$ 
5   if  $piece$  is white then
6      $idx \leftarrow i \oplus 56$  // mirror for white
7      $mg[W] \leftarrow mg[W] + MG\_PSQT(piece, idx) + MG\_MAT(piece)$ 
8      $eg[W] \leftarrow eg[W] + EG\_PSQT(piece, idx) + EG\_MAT(piece)$ 
9   else if  $piece$  is black then
10     $idx \leftarrow i$ 
11     $mg[B] \leftarrow mg[B] + MG\_PSQT(piece, idx) + MG\_MAT(piece)$ 
12     $eg[B] \leftarrow eg[B] + EG\_PSQT(piece, idx) + EG\_MAT(piece)$ 
13  $mgPhase \leftarrow \min(24, phase)$ 
14  $egPhase \leftarrow 24 - mgPhase$ 
15 Compute  $MG(P)$  and  $EG(P)$  as difference between side-to-move and opponent
16  $Eval(P) \leftarrow (MG(P) \cdot mgPhase + EG(P) \cdot egPhase) / 24$ 
17 return  $Eval(P) + 20$  // tempo
```



— 4.4.7 Limitations of the Evaluation

The evaluation is intentionally simple and focuses on material and PSQT-based positional features. This makes it fast and easy to analyze, but it also implies several limitations:

- **Limited feature set:** Pawn structure (isolated/doubled/passed pawns), mobility, rook activity on open files, bishop pair, and advanced king safety concepts are not modeled explicitly.
- **No explicit drawishness detection:** The evaluation does not implement specialized scaling for fortress-like positions, opposite-coloured bishop endgames, or other draw-prone material configurations.
- **Constant tempo term:** The tempo bonus is a fixed constant and does not depend on phase or position characteristics.
- **No incremental evaluation:** The score is computed by scanning all 64 squares, rather than updating evaluation terms incrementally during make/unmake.

The first of these is the one that turns out to matter most in practice. Since neither passed pawns nor pawn structure are represented, two moves that leave material untouched can be scored almost identically even when one wins and the other draws. Section 5.7.1 shows a position in which exactly this happens: both candidate moves are pawn pushes on the same file, and the evaluation has no term that could prefer the one advancing a passer over the one advancing a pawn into its own blockade.

Despite these limitations, the evaluation is sufficient for the primary focus of this thesis, namely the investigation of search behavior and iterative deepening under time constraints. A stronger evaluation can be integrated later without changing the surrounding search architecture.



5 Experimental Evaluation

This section presents the experimental evaluation of the implemented chess engine. The evaluation pursues the central question of this thesis: where lies the sweet spot between invested search time and decision quality, and how do the individual search refinements shift this trade-off? The strongest empirical effect is produced by null-move pruning, which is therefore examined in detail, while the Principal Variation Search parameter sweep yields a deliberate null result that is reported and explained rather than hidden. All measurements are based on a common position suite and a uniform scoring methodology, so that the individual experiments remain directly comparable.

5.1 Research Objectives

The experimental evaluation addresses the following research questions:

The experiments group around three questions. The first concerns the relationship between computation and decision quality, the second the effect of the individual search refinements, and the third the origin of the errors that remain.

Time and decision quality.

- **Time-to-quality and depth-versus-time:** how do reachable depth and move quality develop as the time budget increases, and from which budget onward is no further improvement detectable?
- **Move stability:** at which search depth does the decision settle, measured both by the identity of the chosen move and by its distance to a reference evaluation?

Effect of the search refinements and their resources.

- **Null-move pruning:** how does it affect reachable depth, node throughput, transposition table behavior, and move quality across time budgets?
- **Search tree reuse:** how much depth does a persistent transposition table gain over the course of a game compared to starting each search from an empty table?
- **Principal Variation Search:** how much depth does scouting gain over a plain full-window search, and does that gain depend on the choice of its two parameters?
- **Transposition table size:** from which table size onward does the available memory stop limiting the search? This calibrates the configuration under which the other experiments run.

Origin of the residual error.

- **Distribution of move quality:** how is the error spread over the suite, and which positions account for it?

In addition, the engine is compared against a fixed reference opponent in a match. That experiment is a sanity check on the implementation as a whole rather than a research question in its own right, since its result depends entirely on the chosen opponent configuration.

5.2 Experimental Setup

Unless stated otherwise, all experiments share the following configuration.

Hardware and Build. All measurements were conducted on a MacBook Pro with an Apple M3 Pro processor and 18 GB of RAM. The engine was compiled in release mode and executed as a single-threaded process. Apart from the parameter that is varied in a given experiment, all engine options were kept at their defaults throughout.

Position Suite. The experiments use a fixed suite of 309 positions, stored in the repository as `tests/data/stockfish-bench-all.epd` and derived from the position set that Stockfish uses for its own benchmark. Each configuration is measured with three repetitions per position to average out scheduling noise, and reported figures are means over positions and repetitions.

The suite was chosen because it is public, fixed and therefore reproducible, but its composition has to be kept in mind when reading the results. It contains between 5 and 32 pieces, with a median of 19; by piece count, 25.8 % of the positions are opening-like (≥ 26 pieces), 34.0 % middlegame-like, and 40.2 % endgames (fewer than 16 pieces). The positions are also not balanced: the median reference evaluation is 85 cp, only 11 % of the positions are approximately equal ($|\text{eval}| \leq 50$ cp) and 16 % are already decided ($|\text{eval}| > 300$ cp). This is a benchmark set built to exercise a search, not a sample of the positions an engine faces in a game, and endgames in particular are over-represented relative to ordinary play. All results in this chapter therefore describe the behavior of the engine *on this suite*; generalizing them to tournament play would require a second, game-derived position set, which is left to future work.

Time Budgets. Time-dependent experiments sweep a fixed set of per-move budgets of 10, 25, 50, 100, 250, 500, 1000, 2000, and 5000 ms. This range spans from very short thinking times, where only shallow searches are possible, to budgets that reach depths well into the middlegame.

Quality Scoring. Because the positions have no known ground-truth best move, move quality is measured against a strong reference. For every position, Stockfish is run to a fixed depth of 20, single-threaded, with its default hash size and without tablebases, to obtain a reference best move and a reference evaluation. The move the engine actually played is scored by applying it and analysing the resulting position to the same depth, again from the perspective of the side that moved, so that both values are produced under identical conditions and remain comparable. Mate scores are first converted to a large finite value, and the resulting centipawn loss is then clamped at 1000 cp so that a single decided position cannot dominate an average. The clamp applies to *every* loss above that threshold, not only to those involving a mate. At a one-second budget it is reached in three of the 309 positions: in two of them either the reference move or the played move leads to a forced mate, while in the third both evaluations are ordinary and the raw difference is 1165 cp. All aggregate figures in this chapter are therefore mildly conservative, since these three losses enter the statistics smaller than they are. Two metrics are derived from this reference:



- **Centipawn loss (CPL):** the difference, in centipawns and from the perspective of the side to move, between the reference evaluation of Stockfish’s preferred move and the reference evaluation of the move actually played by the engine. Lower is better; a value of 0 means the engine played a move of equal reference value.
- **Agreement:** the fraction of positions in which the engine’s chosen move coincides exactly with Stockfish’s reference move.

CPL is the primary quality metric, as it also credits moves that differ from the reference move but are of comparable value; agreement is reported as a complementary, stricter indicator. Figure 8 traces the procedure for one concrete position. Note that the metric never asks whether a move is “correct” in any absolute sense; it only asks how much worse the position becomes compared to the best available alternative, which is what makes small inaccuracies and outright blunders comparable on a single scale.

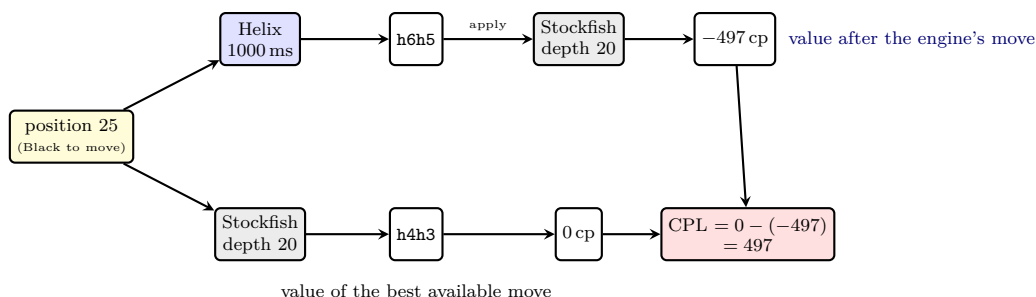


Figure 8: How a move becomes a number, shown for position 25 of the suite. The same position is given to the engine under its time budget and to the reference engine without one. The engine’s move is applied and the resulting position is evaluated to the same fixed depth, so both values describe the same quantity under identical conditions. Their difference is the centipawn loss: here the reference holds a balanced position at 0, while the engine’s move leaves it roughly five pawns worse, giving a loss of 497 cp. A value of 0 means the engine’s move is as good as the reference move, not that it is the same move.

Since the dependent variable of every quality statement in this chapter is produced by this reference, its stability is a limitation of the setup. Depth 20 was chosen as a compromise between reference strength and the cost of scoring several thousand positions, but it was not validated against a deeper reference. A sensitivity check, rescoring the suite at a substantially higher depth and comparing how often the reference move and the resulting CPL change materially, would be needed to rule out that individual conclusions depend on this choice.

Benchmark Harness. All experiments are driven by the dedicated benchmarking harness in the `bench/` directory, which records for each search, among others, the completed depth, selective depth, node and quiescence-node counts, the number of PVS re-searches and null-move cutoffs, transposition table statistics, and the elapsed time. The raw per-search records are written to CSV files and aggregated offline, which keeps measurement and analysis reproducible. Each file also stores the commit hash of the engine it was produced with, so every table in this chapter can be traced back to a specific state of the implementation. All measurements reported here were produced with a single build, commit `f3270d5` of the engine repository¹, so later changes to the engine do not affect the figures given in this chapter.

¹<https://github.com/RealMarvn/Helix>

Separate Runs. The individual experiments were executed as separate runs, so the same configuration can appear with slightly different absolute numbers in different tables; at a one-second budget the completed depth of the default configuration differs by a few hundredths of a ply between runs. Comparisons are therefore always drawn within a run, between configurations measured against each other, never across tables.

Uncertainty. The dominant source of variation in these experiments is not measurement noise but the difference between chess positions: the same configuration can lose zero centipawns in one position and several hundred in another. Repetitions alone therefore say little about how reliable a reported difference is. Since almost all comparisons are paired, the same positions are measured under both configurations, uncertainty is quantified on the paired differences instead. For each position i the three repetitions are averaged first, $x_i = \frac{1}{3} \sum_r x_{ir}$, and the paired difference $d_i = x_{i,A} - x_{i,B}$ is formed from these per-position means; the bootstrap then resamples the d_i . Reported intervals are 95 % percentile bootstrap confidence intervals over 20 000 resamples, drawn over the unit that is actually independent: positions for the suite experiments, and complete games for the tree-reuse experiment, whose plies are clustered within games. There, the per-ply differences are first averaged within each game and the game means are resampled. Seeds are fixed per reported value, so the intervals are reproducible; the analysis script is included with the implementation. An interval that excludes zero is reported as a difference; one that does not is reported as no detectable difference, which is not the same as evidence that no effect exists.

Correctness of the Move Generator. All results rest on the assumption that the engine generates and applies moves correctly. This is verified with *perft* [2], the standard correctness test for chess engines: for a set of reference positions, the number of leaf nodes of the full game tree is counted up to a fixed depth and compared against published values. Any error in move generation, in make-unmake, or in the handling of castling, en-passant and promotion changes these counts and is thus detected. The engine reproduces the reference values for all tested positions and depths.

5.3 Playing Strength (Engine versus Engine)

As an external sanity check on the implementation, the engine is evaluated in head-to-head matches against a fixed reference opponent. This experiment does not place the engine on an absolute rating scale, it only relates it to one specific opponent configuration. The matches are played through the CuteChess framework, with a development build of Stockfish as the reference opponent. To obtain a comparison in the target strength range, Stockfish is restricted through its built-in strength limitation (`UCI.LimitStrength` with a nominal target of 2000 Elo), a single search thread, and a 32 MB hash; Helix likewise runs single-threaded with a 32 MB hash and otherwise default settings. Both engines play a blitz time control of five minutes per side with a 0.5 second increment, which is the range in which time management is most visible while still allowing searches of useful depth.

To obtain a fair and diverse sample, the games are opened from a balanced opening book (`8moves_v3`), with each opening played once from either side so that both engines receive identical starting positions as White and Black. In total 400 games are played with alternating colors. Clearly decided games are adjudicated to keep the match tractable: a game is scored as won once both engines report an evaluation beyond ± 600 centipawns for three consecutive moves, and as drawn once the evaluation stays within ± 10 centipawns for eight consecutive moves from move 40 onwards. All games run on the Apple M3 Pro machine described in Section 5.2. Playing strength is reported as the Elo difference derived from the



match result by the testing framework, together with the likelihood of superiority (LOS) and the draw ratio.

Two limitations apply to the reported interval. The games are played as colour-swapped opening pairs, so the 400 games are not 400 independent trials; the interval, computed by the framework under an independence assumption, is therefore optimistic. And the LOS of 100% is a rounded value, not a probability of one.

Table 1 summarizes the match.

Metric	Value
Games played	400
Score (W–L–D)	288–76–36
Elo difference	$+205.0 \pm 37.8$
Likelihood of superiority	100.0 %
Draw ratio	9.0 %

Table 1: Match result of Helix against Stockfish limited to a nominal 2000 Elo, over 400 games at a 5+0.5 time control.

Helix wins the match decisively, with an Elo difference of roughly +205 points and a likelihood of superiority of 100%. The advantage holds with both colors: Helix scores 138–40–22 as White (74.5%) and 150–36–14 as Black (78.5%), so it remains clearly stronger even without the first-move advantage. The moderate draw ratio of 9.0% indicates that most games are decided rather than drawn.

Two aspects of this result deserve interpretation. That the engine performs slightly better as Black is not a property of the engine but of the opponent: Stockfish’s strength limitation introduces random inaccuracies, and a game in which the weakened side moves first offers more opportunity for such an inaccuracy to decide the game early. More importantly, the result contrasts with the position-based measurements that follow. A margin of 205 Elo describes an engine that plays soundly across a whole game, while Section 5.7 shows that it fails badly in a small class of positions. Both are true at once: the failures are concentrated in sparse endgames, and games between engines of this strength difference are usually decided long before such an endgame is reached. Game results and per-position analysis therefore measure genuinely different things, which is the reason this thesis relies on the latter.

It should be emphasized that this is a *relative* result. The reference engine was deliberately weakened, and Stockfish’s `UCI_LimitStrength` provides only a nominal Elo target that does not map directly onto established rating lists; it also weakens play partly by introducing random inaccuracies rather than uniformly. The reported difference therefore characterizes Helix’s strength relative to this specific, equally configured opponent, not an absolute rating.

5.4 Effect of Null-Move Pruning

Null-move pruning (Section 4.2.7) is the search refinement with the largest measured effect on the engine. It was evaluated by running the full time-budget sweep twice, once with pruning enabled and once disabled, on the common suite. Table 2 reports the reachable depth and the centipawn loss for both configurations.

Budget (ms)	Completed depth (ply)			Centipawn loss		
	no NMP	NMP	Δ [95 % CI]	no NMP	NMP	Δ [95 % CI]
10	3.69	3.93	+0.23 [+0.19, +0.28]	60.2	61.9	+1.7 [−1.6, +4.9]
25	4.32	4.68	+0.36 [+0.31, +0.41]	58.9	60.7	+1.8 [−2.1, +5.5]
50	4.89	5.34	+0.44 [+0.39, +0.50]	57.1	56.0	−1.1 [−8.9, +6.7]
100	5.40	6.00	+0.59 [+0.52, +0.66]	57.2	60.2	+3.0 [−5.2, +11.8]
250	6.07	6.85	+0.78 [+0.70, +0.85]	54.6	55.7	+1.1 [−7.5, +8.5]
500	6.56	7.58	+1.02 [+0.95, +1.09]	56.8	55.0	−1.9 [−8.1, +3.8]
1000	7.12	8.21	+1.09 [+1.02, +1.17]	55.5	52.4	−3.1 [−9.3, +2.5]
2000	7.59	8.84	+1.25 [+1.17, +1.33]	49.2	50.3	+1.1 [−9.7, +11.4]
5000	8.14	9.49	+1.35 [+1.27, +1.43]	43.8	44.4	+0.6 [−4.3, +5.1]

Table 2: Reachable depth and centipawn loss with and without null-move pruning, over the 306 positions for which a reference score is available, with three repetitions per budget. Δ is the paired difference (NMP minus no NMP) with a 95 % bootstrap confidence interval over positions; positive values mean more depth respectively higher loss.

The clearest and most robust effect is on search depth: null-move pruning reaches a greater depth at *every* one of the nine budgets, and the gap widens with more time, from +0.23 ply at 10 ms to a full ply (8.21 versus 7.12) at one second and +1.35 ply at five seconds. This confirms that skipping provably weak subtrees lets the engine invest the saved effort into depth.

This additional depth is not bought at the cost of node throughput. At the one-second budget the engine reaches about 1.69 million nodes per second with pruning versus 1.61 million without, so pruning is if anything slightly faster per node as well. What does change is the transposition table hit rate, which drops from 0.208 to 0.158 at one second: by pruning subtrees, the search visits a larger number of *distinct* positions and re-visits known entries less often. The hit rate is therefore best read as a symptom of the search shape rather than as a quality target in itself. The left panel of Figure 9 shows the node throughput across the whole range of budgets.

The effect on move quality does not follow the depth gain. The paired differences scatter around zero: the pruning configuration is ahead at three of the nine budgets and behind at six, the largest single values being −3.1 cp at one second and +3.0 cp at 100 ms. Crucially, the confidence interval contains zero at *every* budget, and the intervals are wide, spanning ten centipawns or more, because the per-position differences vary far more than their mean. The experiment therefore does not show that pruning leaves quality untouched; it shows that any effect is small compared to the spread between positions and cannot be resolved with this sample. The contrast to the depth column, where every interval excludes zero by a wide margin, is what makes the comparison informative. The right panel of Figure 9 plots the loss over the same budgets.

Taken together, this is the central observation of this experiment: null-move pruning reliably buys depth, while no corresponding gain in move quality could be detected. One qualification matters for the interpretation. Null-move pruning does not give the engine the same search plus one exact ply, it changes which subtrees are explored at all; the comparison is therefore between two search algorithms under equal time, not a clean measurement of what one additional ply is worth. Isolating the latter would require the same algorithm run at different fixed budgets. What the experiment does establish is that a refinement

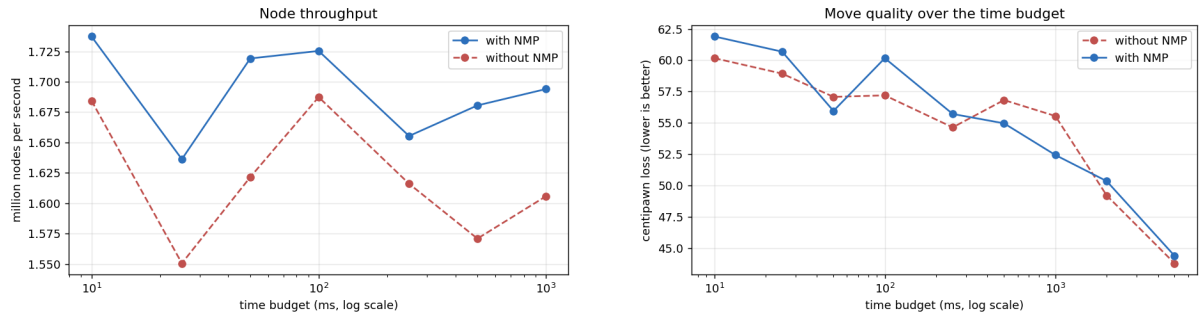


Figure 9: Effect of null-move pruning across the nine time budgets. Left: node throughput in nodes per second. Right: mean centipawn loss. Pruning is slightly faster per node and reaches greater depth at every budget (Table 2), while the two quality curves remain intertwined throughout.

which demonstrably deepens the search does not, on this suite, demonstrably improve its decisions, and sections 5.6, 5.7 and 5.10 examine where that gap comes from.

5.5 Transposition Table Size

The transposition table trades memory for search efficiency: a larger table retains more previously computed positions but consumes more memory. To determine a sensible default, the table size was swept over 1, 4, 16, 64, and 256 MB at a fixed one-second budget, with null-move pruning enabled. Table 3 reports the reachable depth, the eviction rate (the share of stores that overwrite an entry of a different position), and the hit rate.

Table size (MB)	Depth (ply)	Δ to 256 MB [95 % CI]	Eviction rate	Hit rate
1	7.75	-0.435 [-0.498, -0.373]	0.610	0.104
4	8.01	-0.170 [-0.215, -0.128]	0.263	0.134
16	8.14	-0.045 [-0.070, -0.023]	0.076	0.153
64	8.18	-0.003 [-0.014, +0.005]	0.019	0.161
256	8.18	—	0.005	0.163

Table 3: Effect of transposition table size at a one-second budget, over 309 positions and three repetitions. The third column gives the paired difference in depth against the largest table with a 95 % bootstrap confidence interval over positions.

The cost of a small table is substantial and clearly resolved: at 1 MB the search loses 0.44 ply against the largest table, with more than 60 % of all stores evicting a foreign entry, so useful information is displaced before it can be reused. The penalty then shrinks rapidly, to 0.17 ply at 4 MB and 0.05 ply at 16 MB. Both of these differences are still resolved by the data, so the curve does not become flat as early as the raw means suggest; what changes is that the effect becomes practically irrelevant, an order of magnitude below the ply gained by null-move pruning. Only from 64 MB onward is no difference to the largest table detectable at all, even though the eviction rate keeps falling. For the target time controls the search therefore stops being appreciably memory-bound around the 32 MB default used elsewhere in this work.

The shape of this curve follows from what the search actually needs. At a one-second budget the engine visits roughly 1.5 million nodes, and a 1 MB table holds about 32 000 entries, so it is overwritten many times over during a single search: the 61 % eviction rate means most entries are displaced before they can be reused, and the table degenerates from a memory into a cache of the most recent positions. Every additional megabyte then buys fewer new hits than the previous one, because entries are displaced in rough order of usefulness, and once the table is large enough to survive one search the remaining gain comes only from positions that recur rarely. That the curve flattens near the size actually needed for one search, rather than at some arbitrary point, is what makes the result transferable: an engine searching ten times as many nodes would need a correspondingly larger table.

5.6 Time-to-Quality and Depth-versus-Time

The per-budget measurements in Table 2 also answer the time-to-quality and depth-versus-time questions directly. Reachable depth grows smoothly and predictably with the budget: a log-linear fit over all nine budgets yields 0.63 additional plies per doubling of time ($R^2 = 0.999$), and the individual doublings stay within a narrow band around it, between +0.63 and +0.72. Inverted, this corresponds to an effective branching factor of $2^{1/0.63} \approx 3$, meaning each additional ply costs about three times the work of the previous one. This is the exponential relationship discussed in Section 3, and it is the reason the budget has to grow by a factor of roughly 500 to buy the five and a half plies separating the smallest budget from the largest.

Move quality, in contrast, improves far less smoothly. Over the whole range from 10 ms to 5000 ms, a factor of 500 in thinking time, the mean centipawn loss of the pruning configuration falls only from about 62 to 44 cp, and it does so non-monotonically: at 100 ms the curve even rises by 4 cp against the preceding budget before continuing downwards. No single doubling of the budget produces a difference whose confidence interval excludes zero; only comparisons spanning several steps do, for instance from 500 ms to 5000 ms.

To avoid identifying a saturation point by eye, it is defined here explicitly. Call a budget t *saturated* if no larger budget yields a paired improvement whose 95 % confidence interval excludes zero, and let t^* be the smallest budget that is saturated and stays saturated:

$$t \text{ saturated} \iff 0 \in \text{CI}_{95}(\text{CPL}(t') - \text{CPL}(t)) \quad \text{for all } t' > t,$$

$$t^* = \min\{t : \text{every } s \geq t \text{ is saturated}\}.$$

The second condition is necessary because saturation alone is not monotone: at the shortest budgets the per-position differences scatter so widely that nothing is detectable there either, which would otherwise mark 50 ms as a saturation point. On this suite the criterion gives $t^* = 1000$ ms. At 500 ms a further improvement towards 5000 ms is still detectable, at one second it is not. The median tells the same story more directly: it falls from 16 cp at 10 ms over 7 cp at 100 ms to 2 cp at one second, and then stops, standing at 3 cp at five seconds.

Two qualifications belong to this number. First, t^* marks the point where further improvement is no longer *detectable* with this sample, not where it provably ceases; the intervals are wide, so a small effect could remain hidden. Second, mean and median diverge here in a way that matters: while the median stops falling at one second, the mean still decreases from 52 to 44 cp between one and five seconds, because a minority of positions continues to improve. Section 5.7 shows that this minority dominates the mean. The defensible formulation is therefore that no further improvement of *typical* move quality is detectable beyond roughly one second on this suite, while *average* quality keeps improving through a small group of



difficult positions.

5.7 Distribution of Move Quality

All centipawn losses reported so far are means over the suite, and that summary turns out to hide the actual behavior of the engine. The distribution of the per-position loss is strongly skewed: at a one-second budget the *median* loss is 2 cp while the *mean* is 52.4 cp. In 49.0 % of all positions the engine's move loses nothing at all against the reference: in 43.8 % it is exactly the move Stockfish prefers, and in the remaining 5.2 % a different move of equal reference value. In 66.3 % of the positions the loss stays within 25 cp. At the other end, 6.9 % of the positions (21 of 306) exceed 200 cp, and the fifteen worst positions alone account for 46.3 % of the entire mean. The average is therefore not a description of the typical case, it is a description of a small number of failures.

This also changes how the time-to-quality result of Section 5.6 should be read. Measured by the median, additional time helps clearly and monotonically: the median loss falls from 16 cp at 10 ms over 7 cp at 100 ms to 2 cp at one second. The mean, dominated by the outliers, barely moves over the same range. The engine does not gradually get better at every position, it solves more and more positions outright while a hard residue stays unsolved regardless of the budget.

The outliers share a recognizable profile. Positions with a loss above 200 cp hold on average 13.8 pieces, against 19.5 for the well-played ones, and a third of them are endgames with ten pieces or fewer. Three positions reach the scoring cap of 1000 cp, meaning Stockfish sees a forced mate the engine misses entirely.

— 5.7.1 The Worst Positions in Detail

Since a handful of positions determines the headline figure, it is worth naming them. Table 4 lists the ten positions with the highest loss at a one-second budget, together with the move the engine chose and the move the reference preferred.

#	Position (FEN)	Engine	Reference	CPL
22	8/1p3pp1/7p/5P1P/2k3P1/8/2K2P2/8 w - - 0 1	g4g5	f5f6	1000
46	r2r1n2/pp2bk2/2p1p2p/3q4/3PN1QP/2P3R1/P4PP1/5RK1 w - - 0 1	c3c4	g4g7	1000
202	5b2/5k2/R1K5/2p1P1B1/p2r1PP1/4r3/8/7R b - - 8 50	a4a3	f8g7	1000
21	8/2p5/8/2kPKp1p/2p4P/2P5/3P4/8 w - - 0 1	e5f5	d5d6	568
25	5k2/7R/4P2p/5K2/p1r2P1p/8/8/8 b - - 0 1	h6h5	h4h3	497
203	8/R4kb1/2K5/2p1P1B1/p2r1PP1/4r3/8/7R b - - 10 51	f7g8	f7g6	432
307	8/8/5pk1/7p/7P/2b1K3/R4N1r/8 b - - 0 63	h2h4	c3e1	367
240	1r2qbk1/2Rn2p1/r2Ppp1p/1pB5/1P2P3/1Q3N1P/p4PP1/3R2K1 w - - 0 38	b3c3	d1a1	363
148	8/3k4/4p1R1/2P1K1P1/8/2P1N3/8/6rb w - - 11 46	g6e6	e5f4	333
268	2r1r1k1/1p3ppp/p2P1p2/2P5/2q2P2/P1bb1NN1/R4QPP/2R3K1 b - - 0 24	c8c5	e8d8	331

Table 4: The ten positions with the highest centipawn loss at a one-second budget, with the complete FEN of each. The three values of 1000 are all clamped: in positions 46 and 202 a forced mate is involved, in position 22 the raw difference is 1165 cp between two ordinary evaluations.

Before examining individual cases, it is worth noting how the ten distribute. Five of them are endgames with at most eight pieces beside the kings, two are late middlegames, and three are full middlegames; two contain a forced mate the engine does not find (positions 46 and 202). One property holds without exception: in every one of the ten the reference move is a *quiet* move, and in four of them the engine answers with a capture instead. The failures are therefore not scattered but fall into three recurring mechanisms, and the three positions examined below, shown in Figure 10, are chosen as representatives of these rather than as isolated curiosities: a plan that requires more depth than the budget allows (position 22), a tactical resource that move ordering never brings to the front (position 46), and a decision the evaluation cannot make at any depth (position 25).



Figure 10: The three positions discussed below. Left: position 22, White to move, the engine plays **g4g5** instead of the breakthrough **f5f6**. Middle: position 46, White to move, the engine plays **c3c4** and misses the mating sacrifice **g4g7**. Right: position 25, Black to move, the engine plays **h6h5** and gives up the draw held by **h4h3**.

Position 22: a pawn breakthrough that is never seen. White has pawns on f2, f5, g4 and h5 against b7, f7, g7 and h6, with the kings on c2 and c4. White's h-pawn is blocked by the black pawn on h6, so the position is only winnable by opening that file. The reference move **f5f6** sacrifices a pawn to deflect the g7 pawn, but its point emerges only two moves later: after **f6 gxf6**, White must first play the quiet **f2f4** to bring up the reserve pawn, so that the breakthrough **g4g5** can be met by **f4xg5** *whichever* pawn Black captures with. Both capture orders transpose into the same position, in which the h-pawn is finally unblocked and promotes in two moves, while Black's counterpart needs four and is caught by White's king. Played immediately, without the preparation, **g4g5** fails to **f6xg5**, after which White has no passed pawn at all.

The engine plays **g4g5** at every budget from 50 ms upwards, and the evaluation swings from +507 to -658 cp: this is not a missed win but a lost game. Three properties of the implementation combine here, and each alone would suffice to miss the line. White is a pawn down for its entire length, and an evaluation built on material and piece-square tables sees no compensation for that. The decisive move **f2f4** is quiet, so the quiescence search, which extends captures and en passant only (Section 4.2.6), never looks at it; note that admitting promotions to the extension, the obvious suspect, would not help here either, since every move of the plan up to the promotion is quiet. And the promotion that justifies the whole operation lies thirteen plies from the root, whereas the engine reaches depth 8 at a one-second budget. This position is also the one case in the suite where the reported loss of 1000 is a clamp rather than a mate: the raw difference is 1165 cp.



Position 46: a mating attack that is missed. The reference plays `g4g7`, moving the queen to an empty square directly in front of the black king. Despite appearances this is not a sacrifice: no black piece can capture on `g7`. The check leaves Black exactly one legal reply, `Ke8`, after which the reference reports a forced mate. The engine plays `c3c4` instead. This is not an evaluation problem at all, since a mate is a purely tactical fact that any evaluation scores correctly once it is found; the engine simply never searches far enough along this line. Move ordering is the plausible reason, and in a direction that is easy to overlook. `Qg7+` is a *quiet* move, so MVV-LVA never applies to it and it is sorted behind every capture in the list (Section 4.2.10). Only the killer and history heuristics could lift it, and both require the move to have proved itself elsewhere in the tree first, which is exactly what does not happen here. The position illustrates that good move ordering is not merely a speed optimization: at a fixed budget it decides which moves are examined at all.

Position 25: a draw turned into a loss. Black is nominally a pawn up (rook and three pawns against rook and two), yet the reference evaluates the position as exactly balanced, because White's pawn on `e6` and the active rook on the seventh rank compensate. Black's two h-pawns are doubled, on `h6` and `h4`, and the whole assessment hinges on which of them moves. The reference plays `h4h3`, advancing the front pawn, which is three moves from promotion and forces White to attend to it. The engine plays `h6h5`, advancing the rear pawn directly behind its own, where it is immediately blocked and accomplishes nothing; the evaluation drops by roughly five pawns.

Both moves are pawn pushes on the same file and neither changes material by a single point, so no material-based term can tell them apart. Distinguishing them requires knowing that one pawn is a passer close to promotion and the other is blocked by its own colleague, and the evaluation encodes neither passed pawns nor pawn structure (Section 4.4.7). This is the cleanest example in the suite of a decision that additional search depth cannot repair, because the engine would still be comparing the two resulting positions with an evaluation that scores them almost identically.

A pattern across the outliers. These cases are not isolated. Comparing the 21 positions above 200 cp with the 203 played well (≤ 25 cp) reveals a systematic difference in what the engine chooses: in the outliers it plays a capture where the reference does not in 19 % of cases, against 4 % in the well-played group. The reference, in turn, moves its king in 24 % of the outliers but only 13 % elsewhere. The engine thus errs in a specific direction. It takes material where a positional move is required, and it overlooks king activity, the central resource in sparse endgames. Both follow directly from an evaluation built on material and piece-square tables, in combination with a quiescence search that resolves captures and nothing else.

The pattern is consistent with the limitations of the evaluation function discussed in Section 4.4.7. Material and piece-square tables describe a middlegame reasonably well, but they encode nothing about passed pawns, opposition or king activity, the very concepts that decide sparse endgames. That the affected positions are predominantly sparse, and that they are largely the same ones which never stabilize in Section 5.10, supports this reading, though the present experiments cannot separate it from search-side explanations such as horizon effects. Under either reading, the concentration of the error explains why an across-the-board depth gain such as the one from null-move pruning need not show up in the mean: the positions where additional depth would have to make the difference are precisely the few that dominate it.

5.8 Effect of Principal Variation Search

Principal Variation Search is evaluated in two steps: first against a search that does not scout at all, then across its own parameters. The first comparison answers what the technique contributes, the second whether that contribution depends on how it is tuned.

For the first, scouting was disabled entirely, so that every move is searched with the full window $[\alpha, \beta]$, and the time-budget sweep was repeated with null-move pruning active in both configurations. Table 5 reports the result.

Budget (ms)	without PVS	with PVS	Δ depth [95 % CI]
10	3.86	3.90	+0.040 [+0.010, +0.071]
25	4.54	4.64	+0.099 [+0.061, +0.138]
50	5.20	5.29	+0.094 [+0.054, +0.135]
100	5.80	5.96	+0.163 [+0.119, +0.209]
250	6.67	6.80	+0.131 [+0.091, +0.174]
500	7.33	7.52	+0.187 [+0.143, +0.232]
1000	7.92	8.16	+0.234 [+0.186, +0.284]

Table 5: Completed depth with and without Principal Variation Search, null-move pruning enabled in both cases, over 309 positions and three repetitions. Δ is the paired difference with a 95 % bootstrap confidence interval over positions.

Scouting gains depth at every budget, and every interval excludes zero. The gain grows with the budget, from +0.04ply at 10 ms to +0.23ply at one second, which is roughly a fifth of what null-move pruning contributes at the same budget. The mechanism is visible in the node counts: with scouting the engine examines about 2.5 % *more* nodes per second at one second (1.69 against 1.64 million), because a null-window search produces cutoffs more cheaply than a full-window one. Scouting therefore does not reduce the tree at equal depth so much as it makes each node cheaper, which under a fixed clock converts into additional depth. The transposition table hit rate rises alongside it, from 0.124 to 0.158.

The comparison is limited to reachable depth. Since this experiment records no reference scores, it does not establish whether the additional depth also improves move quality, and Section 5.4 suggests that such a transfer should not be assumed.

— 5.8.1 Parameter Sensitivity

Given that scouting helps, the question is whether its two parameters matter: the minimum scouting depth (`PvsMinDepth`) and the number of leading moves searched with the full window (`PvsScoutAfterMove`). To find out whether tuning them is worthwhile, a full grid of $6 \times 5 = 30$ configurations (`PvsMinDepth` 1 to 6, `PvsScoutAfterMove` 1 to 5) was evaluated on the full suite at a one-second budget with null-move pruning enabled. Table 6 lists a representative selection.

The sweep is a negative result: across all 30 configurations the completed depth stays between 8.09 and 8.16ply and the node count between 1.52 and 1.55 million, a spread of about 1.8 %. Paired against the default configuration over positions, the most aggressive setting differs by +0.002ply, 95 % CI



PvsMinDepth	PvsScoutAfterMove	Depth (ply)	Nodes	Re-searches
1	1	8.16	1 534 720	104.1
2	1	8.15	1 531 145	52.2
3	1	8.16	1 539 544	29.2
6	5	8.09	1 531 458	3.2

Table 6: Representative configurations from the PVS parameter sweep (one-second budget). The default configuration is `PvsMinDepth = 2`, `PvsScoutAfterMove = 1`.

$[-0.003, +0.009]$, and even the most conservative one, which almost disables scouting, by only -0.064 ply, 95 % CI $[-0.096, -0.033]$. The latter interval does exclude zero, so the sweep does detect a difference, but one two orders of magnitude below the ply gained by null-move pruning. No setting offers a practically relevant improvement over the default.

One limitation applies to this sweep: with 30 configurations compared against one another, some differences of this size are expected by chance alone, and no correction for multiple comparisons was applied. The result should be read as the absence of a usable tuning gain, not as a precise ranking of the configurations.

Together with Table 5 this yields a two-part answer. Scouting itself contributes measurably, but how aggressively it is applied does not: the decision that matters is whether to scout at all, not where to start.

The reason is visible in the re-search column. The whole point of the scouting parameters is to control the trade-off between the savings of the cheap null-window scout and the cost of the occasional full-window re-search when the scout fails high. In this engine that trade-off barely materializes: across the whole grid the average number of re-searches per search ranges from about 3 to about 146, against roughly 1.5 million nodes visited, so even in the busiest configuration fewer than one node in ten thousand is a re-search. The existing move ordering (transposition-table move, MVV-LVA captures, killer moves, and the history heuristic) is already good enough that the first move is almost always the best, so the null-window scout almost always holds and there is little re-search cost left for the parameters to tune away. As a side observation, the transposition table hit rate correlates weakly with the number of re-searches (0.158 at 104 re-searches versus 0.144 at 3), which again confirms that the hit rate reflects the shape of the search rather than being an independent quality target. Consequently, the default configuration (`PvsMinDepth = 2`, `PvsScoutAfterMove = 1`) is retained, as no setting in the sweep improves upon it.

5.9 Search Tree Reuse

The second refinement of this thesis is the reuse of previously explored search trees through a persistent transposition table (Section 4.2.9). Its value cannot be measured on isolated positions, since it only materializes when consecutive searches are related, so this experiment replays complete games instead. The 30 games are taken from the match of Section 5.3, and every position of every game is searched with a fixed budget of 500 ms, once in each of two modes: in *keep* the table survives the whole game, which is the engine’s normal behavior, and in *clear* it is wiped before every single search. Both runs see identical positions under identical budgets, so the difference between them is precisely the value of resuming the old tree. Table 7 reports the aggregate over all 2256 plies.

Mode	Completed depth (ply)	TT hit rate	Nodes
TT cleared each move	7.23	0.136	758 979
TT kept (tree reuse)	7.52	0.152	772 368

Table 7: Effect of search tree reuse over 30 complete games (2256 plies) at a 500 ms budget per move.

Keeping the table yields more depth at an unchanged time budget, together with a higher hit rate. The 2256 plies are not independent observations, since they are clustered within 30 games, so the effect is quantified per game: averaging the paired per-ply differences within each game and bootstrapping over games gives +0.254 ply, 95 % CI [+0.218, +0.293], and +0.013 in hit rate, 95 % CI [+0.010, +0.018]. The effect is consistent rather than driven by individual games: all 30 games show a positive mean difference. Treating the plies as independent would have produced a visibly narrower interval and thereby overstated the precision.

A paired comparison per ply shows how the average comes about: in 69 % of all plies both modes reach exactly the same depth, in 28 % the persistent table reaches deeper, and only in the remaining few does the cleared table win. Tree reuse thus does not speed up every single search, it provides a head start in those positions where the new root actually lies inside the previously explored tree.

That reading is supported by the development over the course of a game, shown in Figure 11. The gain grows from +0.16 ply in the opening (plies 0–15) over +0.22 in the middlegame to +0.39 in the endgame (ply 41 onwards), where the hit rate also climbs to 0.207 versus 0.178. This is consistent with a larger share of the stored tree remaining reachable as material decreases, since positions with fewer pieces admit more transpositions. The experiment measures the hit rate, however, not the number of transpositions itself, so this remains an interpretation of the observed pattern rather than a measured mechanism.

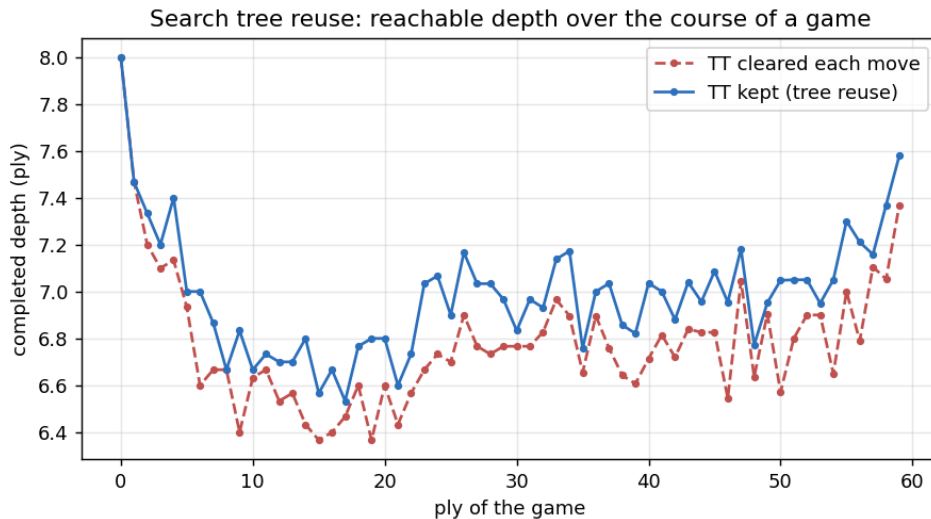


Figure 11: Reachable depth per ply of the game, averaged over 30 games, with the transposition table kept versus cleared before every search.

Compared to null-move pruning, the effect is smaller in absolute terms (+0.25 versus a full ply), but it is obtained differently. Unlike null-move pruning, persistence introduces no additional forward-pruning



heuristic; it reuses bounds and moves that a previous search already computed exactly. Whether this also changes move quality was not measured here, since the experiment records depth and table statistics but does not score the played moves against a reference.

5.10 Move Stability

The final question concerns move stability: at which depth does the engine's chosen move settle, so that deeper search no longer changes the root decision? This is measured with a single iterative-deepening search per position. Because iterative deepening already produces a best move at every completed depth, one search yields the full sequence of root moves without repeated re-searching. Each position is searched under a fixed budget of 20 million nodes with the transposition table cleared beforehand, so that the depth sequence reflects one self-contained search. A node budget is used instead of a fixed target depth because it bounds the cost per position and, being independent of wall-clock time, is unaffected by other load on the machine.

For each position the *stabilization depth* is defined as the smallest depth from which the root move matches the move of the deepest completed iteration and does not change again *within the searched range*. Positions whose move still changes at the deepest searched depth are counted as not yet settled within the budget. Of the 309 positions, 306 produced a usable move sequence, reaching a mean depth of 10.1 plies. Figure 12 shows the cumulative distribution and Table 8 lists selected thresholds.

This definition can only observe what happens inside the budget, which has to be kept in mind when reading the numbers. A move that survives until the last completed iteration might still change one ply deeper, and the evidence for a late stabilization is necessarily thinner than for an early one: a position that settles at depth 3 and is searched to depth 11 confirms that move eight more times, whereas one that settles at depth 10 out of 11 confirms it once. The measurement quantifies this directly. On average a stabilized move is confirmed by 4.3 further iterations (median 3), but for 22.2 % of the positions the move changed in the very last iteration, which is why those are reported as unsettled rather than as stabilized at the final depth. Restricting the analysis to the 178 positions whose move is confirmed by at least three further iterations shifts the median stabilization depth down to 3 plies, with 87 % settled by depth 6. That subset is itself selective, since it excludes late stabilizers by construction, so the two views complement rather than replace one another. Note that the censoring can bias the reported depths in either direction: a move that looks settled may still change beyond the horizon, in which case the true stabilization depth is larger, while restricting the analysis to well-confirmed cases discards exactly the late stabilizers and thus reports smaller depths. All figures below should therefore be read as *observed* stabilization within the searched range, not as the depth ultimately required.

Depth d (ply)	Positions settled by d
2	26.8 %
4	36.9 %
6	52.6 %
8	74.5 %
10	88.9 %

Table 8: Cumulative share of positions whose best move has stabilized by a given depth (306 positions, 20-million-node budget).

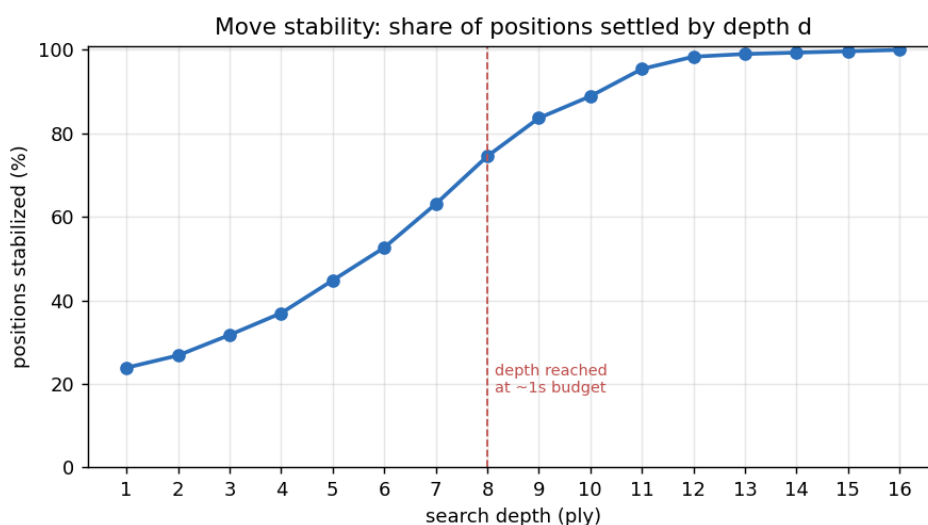


Figure 12: Share of positions whose root move is settled by depth d . The dashed vertical line marks the depth (≈ 8) the engine reaches at a one-second budget.

The root move settles early for most positions: the median observed stabilization depth is 6 plies, and 23.9% of positions never change their move within the searched range at all. By depth 8, roughly the depth the engine reaches at a one-second budget (Table 2), three quarters of all positions have stopped changing their move for the remainder of the search. Within the tested range, additional search beyond this point keeps increasing depth but only rarely changes the decision.

At the same time, the effect is not uniform. Roughly one position in five (22.2%) still changes its move at the deepest depth searched and is thus not observed to stabilize at all. These are the positions whose root decision remains depth-sensitive within the tested range, and they are the only ones for which the extra ply bought by null-move pruning (Section 5.4) can change anything. Since they make up only a fifth of the suite, an across-the-board gain in depth is diluted accordingly, which is consistent with the depth gain of null-move pruning leaving the average centipawn loss unchanged. Decision stability is therefore concentrated: the majority of decisions are made early, while a minority remains sensitive to further search.

— 5.10.1 Stability in Evaluation Rather Than in the Move

Judging stability by the identity of the move has a blind spot: a switch between two moves of equal value counts as instability, while converging on a consistently poor move counts as success. To separate these cases, every move of every iteration was additionally scored against the reference engine, and a position is called ε -stable from the smallest depth at which its centipawn loss stays below ε for all deeper iterations. Table 9 contrasts the two criteria and Figure 13 shows the corresponding curves.

The two criteria disagree in both directions, and the curves cross at around depth 6. Below that, judging by evaluation reports *more* stability: at depth 3 already 39.7% of the positions are within 20 cp, against 31.8% whose move has settled. The engine reaches a decision of adequate quality earlier than it settles on one particular move, since part of the apparent instability is merely alternation between moves of comparable value.



Criterion	Positions reaching it	Median depth	By depth 6	By depth 8
Move identity	100 %	6	52.5 %	74.4 %
$\varepsilon = 0$ cp	52.5 %	4	35.1 %	43.9 %
$\varepsilon = 20$ cp	66.9 %	2	49.5 %	60.0 %
$\varepsilon = 50$ cp	78.4 %	1	63.6 %	72.1 %

Table 9: Move stability judged by move identity versus by evaluation, for 305 scored positions. Percentages refer to the whole set, so positions that never reach the criterion are included in the denominator.

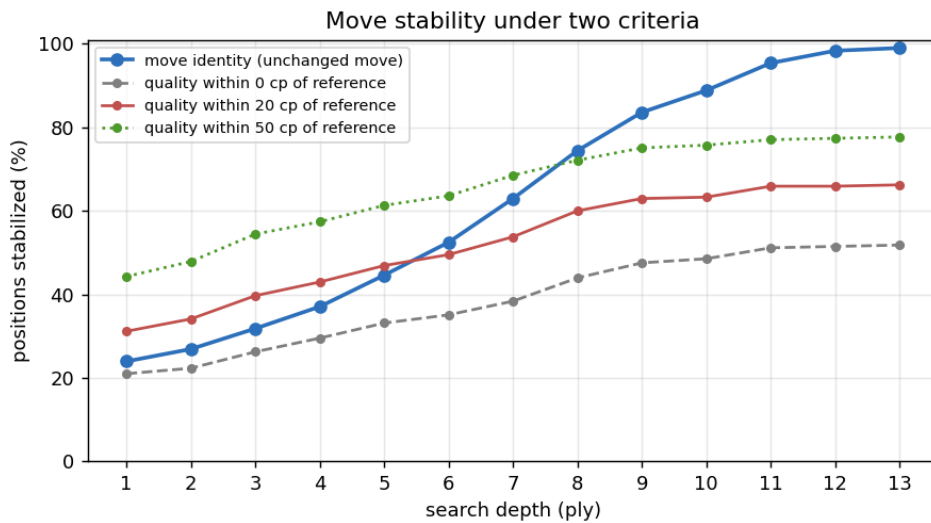


Figure 13: Cumulative share of positions considered stable by depth d , judged by the identity of the move and by its distance to the reference evaluation.

Above depth 6 the relation reverses, and this is the more consequential half. By move identity, 88.9 % of the positions have settled by depth 10; measured against the reference evaluation, only 63.3 % are within 20 cp, and 33.1 % never reach that margin at any depth. These are not positions in which the move keeps changing: their move does settle, but it settles on something poor, with a mean loss of 120 cp at the deepest iteration. Fourteen of the 21 worst positions from Section 5.7 are among them.

Both readings therefore support the same conclusion from opposite sides. Where the engine is able to find a good move, it finds it early, earlier than the move-identity criterion suggests. Where it is not, the search converges confidently on the wrong answer. Stability of the move is thus a necessary but not a sufficient indicator of a settled decision.

It is worth asking whether these positions would simply need more depth. Within the tested range, the data argue against it. Both groups are searched to the same average depth of 10.1 plies under the shared node budget, so the difference is not one of search effort. What differs is how much that effort achieves: comparing the first half of each search to the second, and excluding the final iteration that defines the grouping, the loss falls by 36.5 cp on average for the positions that do converge, but only by 5.4 cp for those that do not. The same picture emerges when comparing shallow to intermediate depths, where the

converging positions gain 41.5 cp and the others 21.9 cp. Additional depth is therefore not without effect on the difficult positions, but it is roughly half to one seventh as effective, and it starts from a worse level.

What this establishes is that the two groups differ in how well they convert depth into quality, not why. The observation is consistent with the evaluation being the binding constraint: the affected positions are materially sparse (Section 5.7), and material with piece-square tables encodes nothing about passed pawns, opposition or king activity, which is exactly what decides such positions. Other explanations cannot be excluded by this experiment, however, among them horizon effects in long forcing lines, the absence of search extensions, or degraded move ordering in sparse positions. Establishing the cause would require an intervention rather than an observation: adding the missing evaluation terms and re-running the affected subset under an identical node budget. If the group improved materially, the argument would be direct. That experiment is left to future work; what can be stated here is that within the depths this engine reaches, the difficult positions do not respond to depth the way the rest of the suite does.



6 Conclusions and Future Work

The experiments of Section 5 allow the central question of this thesis to be answered, and the answer is more nuanced than the initial formulation suggested. This section relates the individual results to that question, draws the overall conclusions, and outlines where the engine would have to be extended for the remaining gap to close.

6.1 Depth and Decision Quality Are Not Proportional

The clearest finding of this work is that additional search depth does not translate into proportionally better decisions. Null-move pruning (Section 5.4) reaches a greater depth at every single one of the nine time budgets, at one second a full ply, with confidence intervals that exclude zero throughout; the corresponding intervals for centipawn loss contain zero at every budget. Search tree reuse (Section 5.9) adds another 0.25 ply, 95 % CI $[+0.22, +0.29]$, without changing any search parameter. Both refinements demonstrably deepen the search, and for neither could a corresponding gain in move quality be detected on this suite. For null-move pruning this follows from the scored comparison directly. For tree reuse, whose experiment records depth rather than reference scores, it follows from the mechanism: the refinement acts solely by making the search reach further, and Section 5.10 shows that additional depth changes the root decision in only a fifth of the positions. A gain of a quarter of a ply is therefore not in a position to move the aggregate, which is exactly what the scored null-move comparison confirms for a gain four times as large.

The move stability measurements (Section 5.10) offer an explanation. By depth 8, the depth the engine reaches at a one-second budget, three quarters of all positions have stopped changing their move within the searched range, and about a quarter never change it from the very first iteration. Deeper search can only change the decision in the remaining fifth, so an across-the-board gain in depth is diluted accordingly. Applying the criterion of Section 5.6, the smallest budget beyond which no further improvement is detectable is one second for this engine and suite.

That number should be read for what it is. It describes typical positions, where the median loss stops falling, while the mean keeps decreasing because a minority of difficult positions continues to improve. And it marks the point where a difference is no longer *resolved* by this sample, not where improvement provably ends: the intervals are wide enough that a small effect could remain hidden.

This should be related to the depth at which the engine operates. Junghanns et al. [6] find that diminishing returns do exist in chess, but that the high error rate of programs conceals them for search depths through nine plies. The budgets used here reach from 3.9 plies at 10 ms to 9.5 plies at 5000 ms and thus lie almost entirely inside that range. The flat quality curve reported above is therefore consistent with the literature rather than in tension with it, and the same fact bounds what it can establish: this setup cannot separate a genuine absence of returns from returns masked by the engine's own error rate.

6.2 Where the Search Is Actually Limited

The distribution of move quality (Section 5.7) shows where the remaining errors come from. The engine plays a move that loses nothing against the reference in 49 % of the positions and stays within 25 cp in

two thirds of them, with a median loss of 2 cp. The mean of 52 cp is produced almost entirely by a small residue: fifteen positions account for nearly half of it.

These positions share a profile. They hold fewer pieces than average, a third of them are endgames, and three contain a forced mate the engine does not find. These are the positions in which material and piece-square tables carry little useful information, since passed pawns, opposition and king activity are not represented in the evaluation (Section 4.4.7). Position 25 of the suite (Section 5.7.1) shows the mechanism in its purest form: both candidate moves are pawn pushes on the same file, neither changes material by a single point, and separating them requires knowing that one pawn is a passer three moves from promotion while the other is blocked behind its own colleague. Depth cannot repair that decision, because the two resulting positions are scored almost identically no matter how long the engine looks at them. The observation is therefore consistent with evaluation quality being the binding constraint there, although the experiments presented here cannot separate it from other candidates such as horizon effects or missing search extensions.

Judging stability by evaluation instead of by the identity of the move makes the consequence explicit (Section 5.10.1). While the move settles in nearly every position, a third of them never come within 20 cp of the reference at any depth, ending at an average loss of 120 cp. The search does not remain undecided in these cases, it converges, and it converges on the wrong move. A stable decision is therefore not by itself a good one, which is worth keeping in mind wherever move stability is used as a stopping criterion in time management.

What can be shown directly is that these positions do not respond to depth the way the rest of the suite does. They are searched just as deeply, to the same average of 10.1 plies, but convert that depth into far less: over the course of a search their loss improves by 5.4 cp where the converging positions gain 36.5 cp. Depth is not useless there, it is merely a much weaker lever against a gap of more than 100 cp. This rules out insufficient search effort as the explanation, but it does not by itself identify the evaluation as the cause; it makes it the most plausible candidate, given which positions are affected. Deciding the question requires an intervention on the evaluation itself, which is the single most valuable experiment this thesis leaves open.

The Principal Variation Search results (Section 5.8) fit the same picture from a different angle. Scouting itself is worth having: it gains 0.23 ply at a one-second budget, with intervals excluding zero at every budget. Its parameters, however, are irrelevant, since the node count varies by less than two percent across all 30 configurations. The reason is that re-searches are rare to begin with: depending on the configuration, between roughly 3 and 146 per search against 1.5 million nodes visited. The move ordering, built from transposition table moves, MVV-LVA, killer moves and the history heuristic, is already good enough that the first move examined is almost always the best one, so the trade-off the parameters are meant to control barely arises. This is consistent with the long-standing observation that the effectiveness of Alpha-Beta pruning is governed by move ordering rather than by the pruning scheme itself [7, 9].

Position 46 marks the boundary of that statement. The move the engine misses there is a quiet queen move giving check, and quiet moves are ranked neither by the transposition table nor by MVV-LVA, so it sits behind every capture until the killer or history heuristic has encountered it elsewhere in the tree. Ordering that is excellent on average can therefore still fail systematically on a particular class of move, and an aggregate over a suite is exactly the wrong instrument for noticing that.

Notably, scouting buys its depth not by shrinking the tree but by making each node cheaper: with scouting the engine visits more nodes per second, not fewer. All three refinements studied here therefore increase reachable depth by different means, and none of them was shown to improve move quality.



6.3 Implications for Time-Managed Search

For iterative deepening under time constraints [8] these results are encouraging. The anytime property is not merely a safety net: since most positions settle early, an interrupted search usually returns the same move a deeper one would have confirmed. Stopping between iterations therefore costs far less quality than the depth difference alone would suggest.

The measurements also show that the engine stops being appreciably memory-bound around the default hash size (Section 5.5). A small table costs real depth, but the penalty falls below a twentieth of a ply by 16 MB and is no longer detectable at 64 MB. Set against the tree-reuse result, this suggests that keeping the table alive across moves is worth more than enlarging it: retaining what was already computed gains 0.25 ply, whereas quadrupling the table from 64 MB upwards gains nothing measurable.

Against a fixed reference opponent, the engine wins its match by roughly 205 Elo points with both colors (Section 5.3). This is a check that the implementation performs sensibly as a whole, not a rating: the figure describes the relation to one weakened Stockfish configuration and would change with any other opponent.

6.4 Limitations

Several limitations qualify these conclusions. Move quality is measured against a strong reference engine rather than against ground truth, so agreement with Stockfish is treated as a proxy for correctness; positions in which a different move is equally good are penalised by the agreement metric, which is why centipawn loss is used as the primary measure.

The suite consists of 309 predefined positions with three repetitions each. It is large enough for the aggregate effects reported here, but the skewed distribution of the loss means that individual means are sensitive to a small number of positions, and differences of a few centipawns should not be over-interpreted.

The stability results are bounded by the observation horizon. A move counts as stabilized when it stops changing within a 20-million-node search, which cannot say what a deeper search would do; roughly a fifth of the positions still change their move at the deepest iteration reached. The comparison between converging and non-converging positions inherits this limitation and is additionally descriptive by construction, since the groups are defined by the very quality behavior whose trajectories are then compared. Defining the groups in advance, by piece count or game phase, would make that comparison independent.

The opponent in the match was deliberately weakened through Stockfish's built-in strength limitation. That mechanism provides only a nominal Elo target and works partly by introducing random inaccuracies, so the reported difference describes the relation to this specific configuration and not an absolute rating.

Finally, the engine uses a mailbox board representation and a handcrafted evaluation, both chosen for clarity rather than for strength. Modern engines rely on bitboards [1] and learned evaluation functions [15], and the conclusions about absolute playing strength do not transfer to those designs.

6.5 Reflection

Three insights from this work were not anticipated when the experiments were designed, and they say as much about the methodology as about the engine.

The first concerns the initial hypothesis. The thesis set out to find the point at which additional time stops paying off, implicitly assuming that time and quality are related by a single curve with a knee. The data show something different: there is no single curve, because positions fall into two regimes. Most are solved early and never benefit from more time, a minority is never solved and benefits just a little. The apparent knee in the aggregate is an artifact of mixing the two. The case studies make the difference tangible: position 22 is solvable by depth in principle, but only thirteen plies from the root, while position 25 cannot be solved by depth at any budget because the evaluation does not distinguish the two candidate moves in the first place. In hindsight the question “where is the sweet spot” was posed one level too coarsely; “for which positions does additional time still help” would have been the more productive formulation from the outset.

The second is methodological. For a long time the analysis reported only mean centipawn loss, and under that view the engine looked mediocre and the refinements looked useless. Only after examining the distribution did it become clear that the median is 2 cp and that half of all positions are answered without any loss against the reference. The mean was not wrong, it simply answered a different question than the one being asked. That a single summary statistic can obscure the central result of an experiment is the most transferable lesson of this thesis, and it is the reason the analysis now reports several estimands side by side.

The third concerns correctness. During the evaluation the engine occasionally produced illegal moves in tournament play, which turned out not to be a search problem at all but a parser that misclassified a knight move onto the en-passant square. The position in the engine and the position in the arbiter drifted apart silently, and every subsequent move was computed for a different game. The measurements taken before that point were internally consistent and looked entirely plausible; nothing in the aggregate numbers indicated a defect. This is the strongest argument for the correctness infrastructure described in Section 5.2: perft validation and the systematic comparison against a reference detect what plausible-looking averages do not. Were the project to be repeated, that infrastructure would come first, before any measurement.

6.6 Future Work

The results point to one clear priority, and it takes the form of a testable hypothesis rather than a conclusion. The static evaluation is the most plausible candidate for the residual error and therefore the most valuable next intervention: adding passed-pawn, king-activity and pawn-structure terms together with dedicated endgame knowledge, and re-running the difficult subset under an identical node budget. If that group improves materially while the rest of the suite stays unchanged, the interpretation offered here would become direct evidence; if it does not, the cause lies elsewhere in the search, which would be an equally informative outcome. Either way this is the experiment that would turn the present interpretation into a result.

A second direction concerns the measurement itself. A single mean is a poor summary of a distribution this skewed, and the answer is not to remove the extreme cases, which are the engine’s real failures and among its most informative observations, but to report several estimands side by side: the mean as expected loss, the median as typical loss, upper percentiles, the share of positions within a given margin,



and the rate of catastrophic errors. Each answers a different question, and stratifying them by position type would show which refinement helps which class of position instead of averaging the two regimes into one number.

The stability analysis could be extended in the same spirit. Judging convergence by evaluation rather than by the identity of the move already proved more informative here (Section 5.10.1); applying the same criterion to the time-to-quality measurements, and reporting from which *budget* rather than from which depth the loss stays within a margin, would describe the practical behavior of the engine under a clock more directly than the depth-based view can.

Finally, the search refinements evaluated here are only the beginning of what a classical engine typically employs. Late move reductions, aspiration windows and static exchange evaluation all target the same goal of spending effort where it changes the decision, and the move ordering measured here is strong enough that they would likely pay off.

6.7 Summary

This thesis implemented a complete chess engine and used it to examine how invested search time translates into decision quality on a fixed position suite. The central finding is that this translation is not uniform. Additional computation reliably increases search depth, but the value of that depth differs sharply between positions, and averaging over the suite hides two regimes rather than describing one.

The majority of positions reaches a low centipawn loss early and stops changing its move at modest depth; the median loss is 2 cp, and further time changes little. A minority behaves in the opposite way: it accounts for nearly half of the mean loss, keeps improving with additional time, and responds to depth far more weakly than the rest, gaining 5.4 cp over a search where the others gain 36.5. These positions are predominantly sparse endgames.

All three refinements studied deliver what they promise in terms of search. At a one-second budget null-move pruning gains a full ply, Principal Variation Search 0.23 ply and search tree reuse 0.25 ply, each with 95 % intervals excluding zero, and each by a different mechanism: pruning subtrees, making individual nodes cheaper, and retaining what was already computed. For none of them could a corresponding improvement in move quality be detected, which follows from the heterogeneity above: a gain in depth applies to all positions, but only a fifth of them are still in a state where depth changes the decision.

Under an explicit criterion, no further improvement of typical move quality is detectable beyond roughly one second per move on this suite. This is an operational statement rather than a claim that improvement ends there. What the evidence supports, without establishing it conclusively, is that beyond this point the limit lies less in how far the engine looks ahead than in what its evaluation is able to distinguish when it arrives.



References

- [1] Chess Programming Wiki. Bitboards. <https://www.chessprogramming.org/Bitboards>. 14, 51
- [2] Chess Programming Wiki. Perft. <https://www.chessprogramming.org/Perft>. 34
- [3] Chess Programming Wiki. PeSTO's evaluation function. https://www.chessprogramming.org/PeSTO%27s_Evaluation_Function. 26
- [4] D. R. Ferreira. The impact of the search depth on chess playing strength. *ICGA Journal*, 36(2):67–80, 2013. 4
- [5] G. Goetsch and M. S. Campbell. Experiments with the null-move heuristic. In *Computers, Chess, and Cognition*, pages 159–168. Springer, New York, 1990. 4, 20
- [6] A. Junghanns, J. Schaeffer, M. Brockington, Y. Bjornsson, and T. Marsland. Diminishing returns for additional search in chess. In H. J. van den Herik and J. W. H. M. Uiterwijk, editors, *Advances in Computer Chess 8*. Universiteit Maastricht, Maastricht, 1997. 4, 49
- [7] D. E. Knuth and R. W. Moore. An analysis of alpha-beta pruning. *Artificial Intelligence*, 6(4):293–326, 1975. 1, 3, 6, 7, 50
- [8] R. E. Korf. Depth-first iterative-deepening: An optimal admissible tree search. *Artificial Intelligence*, 27(1):97–109, 1985. 1, 3, 10, 51
- [9] T. A. Marsland. A review of game-tree pruning. *ICGA Journal*, 9(1):3–19, 1986. 1, 3, 6, 7, 9, 23, 50
- [10] T. A. Marsland and M. Campbell. Parallel search of strongly ordered game trees. *ACM Computing Surveys*, 14(4):533–551, 1982. 3, 8
- [11] R. R. Nair, T. Babu, S. Kishore, K. Pavithra, and S. Sumana. Improving alpha-beta pruning efficiency in chess engines. In *2024 International Conference on IT Innovation and Knowledge Discovery (ITIKD)*, pages 1–6. IEEE, 2025. 3
- [12] J. Pearl. SCOUT: A simple game-searching algorithm with proven optimal properties. In *Proceedings of the First AAAI Conference on Artificial Intelligence*, pages 143–145, 1980. 3, 8
- [13] A. Reinefeld. An improvement to the Scout tree search algorithm. *ICGA Journal*, 6(4):4–14, 1983. 3, 9
- [14] J. Schaeffer. The history heuristic and alpha-beta search enhancements in practice. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 11(11):1203–1212, 1989. 23
- [15] D. Silver, T. Hubert, J. Schrittwieser, I. Antonoglou, M. Lai, A. Guez, M. Lanctot, L. Sifre, D. Kumaran, T. Graepel, et al. A general reinforcement learning algorithm that masters chess, shogi, and Go through self-play. *Science*, 362(6419):1140–1144, 2018. 1, 4, 51
- [16] T. Warnock and B. Wendroff. Search tables in computer chess. *ICGA Journal*, 11(1):10–13, 1988. 3, 15
- [17] G. N. Yannakakis and J. Togelius. *Artificial Intelligence and Games*. Springer, Cham, 2018. 1, 3, 5, 6
- [18] A. L. Zobrist. A new hashing method with application for game playing. *ICGA Journal*, 13(2):69–73, 1990. 3, 15